

UNIVERSITÉ DE TECHNOLOGIE DE TROYES

Respiratory Risk Analytics

Pollution Atmosphérique et Maladies Respiratoires

Auteurs :

Yves APEAPEA-MIGUE
Malcolm ARIDORY
Hélène CHEVALIER
Yesmine FATHALLAH
Gwendal RODRIGUES

Sources de données : EDGAR (Émissions)
GBD 2023 (Maladies)

Période : 1980 – 2022

Couverture : 197 pays

Table des matières

I	Business Understanding	6
1	Contexte Général	6
2	Question Métier et Objectifs	6
3	Enjeux du Projet	7
3.1	Enjeux environnementaux	7
3.2	Enjeux sanitaires	7
3.3	Enjeux économiques	7
3.4	Enjeux sociaux	7
4	Objectifs Métier	7
5	Jeux de Données	8
5.1	Données EDGAR (TG11 – Collecte)	8
5.2	Données IHME : Global Burden of Disease	8
6	Périmètre Analytique	8
7	Parties Prenantes	9
7.1	Commanditaire principal	9
7.2	Équipe projet	9
7.3	Acteurs secondaires	9
8	Risques et Hypothèses	9
8.1	Risques identifiés	9
8.2	Hypothèses	9
II	Data Understanding	10
1	Indicateurs clefs	10
2	Description des Données	11
2.1	Vue d'ensemble (TG12 – Préparation)	11
2.2	Polluants étudiés (EDGAR)	11
2.3	Maladies respiratoires (GBD)	11
3	Qualité des Données	12
3.1	Valeurs manquantes (TG12 – Préparation)	12
3.2	Jointure des données	12
4	Analyse Exploratoire	13
4.1	Distribution des émissions par polluant (TG13 – Traitement)	13
4.2	Comparaison hommes/femmes	14
5	Évolutions Temporelles	15
5.1	Tendances des émissions 1980–2022 (TG23 – Traitement)	15
5.2	Tendances des décès 1980–2022 (TG23 – Traitement)	15
6	Analyse Géographique	16

6.1	Pays les plus émetteurs (TG24 – Restitution)	16
6.2	Pays avec les taux de mortalité les plus élevés	16
6.3	Cartographie des maladies respiratoires (TG24 – Restitution)	17
6.4	Cartographie des émissions de polluants	18
7	Analyse Sectorielle	19
7.1	Secteurs d'activité (TG33 – Traitement)	19
7.2	Relation secteur-polluant (TG34 – Restitution)	20
8	Analyse des Corrélations	21
8.1	Corrélations polluants-maladies (TG13 – Traitement)	21
8.2	Matrice de corrélation complète (TG14 – Restitution)	22
8.3	Relations détaillées – scatter plots (TG14 – Restitution)	23
9	Synthèse de l'Analyse Exploratoire	24
9.1	Qualité des données	24
9.2	Principales conclusions	24
9.3	Recommandations pour la modélisation	24
10	Chaînes de valorisation	24
10.0.1	Chaîne 1 – Analyse des corrélations polluants-maladies	24
10.0.2	Chaîne 2 – Analyse géographique et temporelle	26
10.0.3	Chaîne 3 – Analyse sectorielle des émissions	27
III	Data Preparation	28
1	Jeux de données (rappel)	28
2	Extraction des données	28
3	Prétraitement des données	29
3.1	Fusion des données (TG41-43 – Transformation)	29
3.2	Stockage des données (TG44 – Export)	30
4	Préparation pour la modélisation	31
4.1	Décalage temporel (Lag)	31
4.2	Gestion des valeurs manquantes (TG51 – Préparation ML)	32
4.3	Transformations des features (TG51 – Préparation ML)	32
4.3.1	Transformation logarithmique	32
4.3.2	Normalisation des features	32
4.4	Sélection des features et cibles	33
4.5	Séparation train/test	33
4.6	Implémentations	33
4.7	Analyse des scénarii	33
5	Chaîne de valorisation	34
5.0.1	Chaîne 4 – Préparation et intégration des données	34
6	Synthèse de la préparation des données	35

IV	Modelization	36
1	Réflexions préliminaires	36
1.1	Modèles, machine-learning et deep-learning	36
1.2	Technologies choisies	37
1.3	Choix des variables	37
1.4	Méthodes d'évaluation d'un modèle	37
2	Implémentation itérative des modèles	38
2.1	Modèle de machine learning scikit-learn (TG52 – Modélisation classique)	38
2.1.1	Ridge	38
2.1.2	Forêts aléatoires (Random Forests)	38
2.1.3	Gradient Boosting	39
2.1.4	Modèle retenu	39
2.2	Modèle de deep learning PyTorch (TG53 – Modélisation deep learning)	40
2.2.1	Réseaux de neurones artificiels (ANN)	40
2.2.2	Notre modèle d'ANN	41
2.2.3	Boucle d'entraînement	42
3	Implémentation finale et résultats	43
3.1	Implémentation	43
3.1.1	Implémentation PyTorch : ANN	43
3.1.2	Implémentation scikit-learn : GradientBoostingRegressor	43
3.1.3	Problèmes rencontrés	43
3.2	Résultats (TG54 – Évaluation)	44
3.3	Prédictions (TG61-64 – Simulation de scenarii)	45
3.3.1	Analyse de scenarii	45
3.3.2	Résultats des prédictions – Gradient Boosting	46
3.3.3	Résultats des prédictions – Réseau de neurones (ANN)	46
3.3.4	Interprétation et limites	46
4	Implémentation dans Omniscope	47
4.1	Portage de la pipeline de préparation	47
4.2	Modèles de régression dans Omniscope	48
4.3	Prédictions des scenarii dans Omniscope	50
4.4	Pertinence d'Omniscope pour la modélisation	51
5	Chaînes de valorisation	51
5.0.1	Chaîne 5 – Entraînement et évaluation des modèles prédictifs	52
5.0.2	Chaîne 6 – Simulation de scenarii de réduction des émissions	53
V	Evaluation	54
1	Évaluation des modèles	54
1.1	Principes généraux d'évaluation	54
1.2	Méthodes appliquées dans notre projet	54
1.3	Limites de notre évaluation	54
VI	Deployment	56
1	Déploiement du modèle	56

1.1	Sauvegarde et sérialisation du modèle	56
1.2	Architecture de déploiement	56
1.2.1	API REST	56
1.2.2	Conteneurisation Docker	56
1.2.3	Déploiement cloud	56
1.3	Déploiement de notre modèle	57
VII	Conclusion	58
1	Conclusion générale	58
	Annexes	i
	Références	i
	Ressources	i
	Remerciements	i
	Liste des codes sources	
1	Extraction des données EDGAR avec Polars	29
2	Aggrégation du dataset combiné pour la corrélation	30
3	Analyse des décalages temporels par maladie	31
4	Sortie du script de préparation des données et présentation du dataset final . . .	35
5	Comparaison des modèles scikit-learn	39
6	Comparaison des modèles scikit-learn (sortie de script)	39
7	Notre ANN	41
8	Boucle d'entraînement de notre modèle	42
9	Notre ANN (rappel)	43
10	Modèle scikit-learn final	43
11	Résultats du modèle de régression gradient boosting	44
12	Résultats du modèle ANN	45

Introduction

La pollution atmosphérique constitue aujourd’hui l’un des principaux déterminants environnementaux de santé. Ses effets sur les pathologies respiratoires sont largement documentés, mais leur ampleur et leur dynamique varient selon les territoires, les périodes et les types de polluants.

Le projet **Respiratory Risk Analytics** s’inscrit dans cette perspective. Il vise à mieux comprendre les interactions entre les émissions atmosphériques et l’évolution des maladies respiratoires à l’échelle mondiale. En s’appuyant sur des bases de données reconnues et actualisées, le projet cherche à fournir des éléments factuels pour aider les décideurs à identifier les zones les plus sensibles et à prioriser les actions de réduction de la pollution.

Ce rapport est structuré selon la méthodologie CRISP-DM :

- **Business Understanding** : compréhension du contexte métier, des enjeux et des objectifs du projet
- **Data Understanding** : exploration et analyse des données disponibles
- **Data Preparation** : préparation des données pour la modélisation
- **Modeling** : modélisation des données
- **Evaluation** : évaluation des modèles
- **Deployment** : déploiement des modèles

Objectifs de l’étude

- Comprendre et quantifier la relation entre la pollution atmosphérique et l’incidence des maladies respiratoires
- Identifier les zones prioritaires d’intervention
- Analyser les tendances temporelles des émissions de polluants atmosphériques
- Étudier la distribution géographique des maladies respiratoires
- Identifier les corrélations entre polluants et pathologies
- Préparer les données pour la phase de modélisation
- Modéliser la relation entre la pollution atmosphérique et l’incidence des maladies respiratoires

Première partie

Business Understanding

1 Contexte Général

Ce projet s'inscrit dans le champ de la santé publique et de l'environnement, deux domaines où les interactions entre qualité de l'air et santé humaine constituent des préoccupations majeures. L'augmentation de la fréquence des épisodes de pollution et la progression des maladies respiratoires en font un enjeu prioritaire pour les autorités sanitaires nationales et internationales. Les politiques publiques s'appuient désormais fortement sur les données pour orienter les stratégies territoriales de prévention et d'intervention.

Plusieurs acteurs sont impliqués dans cette dynamique : les collectivités territoriales, les agences de santé publique, les ministères en charge de la santé et de l'environnement, ainsi que les organisations européennes comme la Commission Européenne. Les citoyens, directement exposés aux risques liés à la pollution atmosphérique, représentent la première population bénéficiaire du projet. Dans ce contexte institutionnel riche, le recours à une analyse *data-driven* constitue un outil d'aide à la décision essentiel.

2 Question Métier et Objectifs

L'objectif global du projet Respiratory Risk Analytics consiste à comprendre et quantifier la relation entre la pollution atmosphérique et l'incidence des maladies respiratoires dans le monde. Cette question métier répond à un besoin décisionnel réel : identifier les zones prioritaires d'intervention et mesurer l'impact potentiel ou réel des politiques publiques de réduction de la pollution.

La question centrale formulée est la suivante : « **Quelle est la relation entre la pollution atmosphérique et l'incidence des maladies respiratoires à l'échelle mondiale ?** ».

Plusieurs sous-questions guident cette problématique :

- **Diagnostiquer** : identifier les zones où les émissions et les maladies respiratoires sont les plus élevées
- **Expliquer** : analyser les liens temporels entre variations de pollution et variations de santé
- **Prédire** : estimer l'évolution potentielle des maladies à partir des niveaux de pollution
- **Recommander** : proposer des actions prioritaires

3 Enjeux du Projet

Les enjeux sont multiples et concernent plusieurs dimensions :

3.1 Enjeux environnementaux

La réduction des émissions de polluants constitue un impératif lié à la fois aux réglementations européennes et à la protection de la biodiversité.

3.2 Enjeux sanitaires

La pollution atmosphérique est reconnue comme un facteur aggravant majeur pour des pathologies telles que l'asthme, la bronchopneumopathie chronique obstructive (BPCO) et les infections respiratoires aiguës.

3.3 Enjeux économiques

Une diminution de l'incidence des maladies respiratoires permettrait une réduction des dépenses hospitalières, des coûts de prise en charge et des pertes de productivité liées aux arrêts de travail.

3.4 Enjeux sociaux

Les enjeux sociaux touchent directement la qualité de vie des populations, notamment les plus vulnérables.

Ce projet présente ainsi une forte valeur ajoutée, car il vise à éclairer la décision publique et à cibler les zones où les actions de réduction de la pollution atmosphérique auraient l'effet le plus significatif.

4 Objectifs Métier

Le projet se fixe plusieurs objectifs métier :

1. **Identification des zones prioritaires** : grâce à une démarche de data mining, identifier les zones géographiques où les marges de réduction de la pollution sont les plus fortes et où les interventions publiques peuvent avoir un impact significatif.
2. **Réduction de l'incidence** : proposer des solutions permettant de réduire l'incidence des maladies respiratoires de 5% dans les zones les plus sensibles, et de diminuer de 10% les émissions des polluants les plus dangereux pour la santé.
3. **Identification des polluants critiques** : identifier les polluants ayant l'impact le plus important sur la santé respiratoire à l'échelle mondiale.

5 Jeux de Données

Pour répondre à ces objectifs, deux principaux jeux de données externes ont été identifiés.

5.1 Données EDGAR (TG11 – Collecte)

EDGAR (Emissions Database for Global Atmospheric Research) : Base de données de la Commission Européenne contenant les émissions de polluants atmosphériques par pays, année et secteur d'activité.

Les données se présentent sous forme de fichiers Excel contenant environ 6 000 lignes par polluant. Les variables incluent :

- L'année (numérique)
- Le pays (catégoriel)
- Le secteur d'émission (catégoriel)
- Le type de polluant (catégoriel)
- La quantité d'émissions en gigagrammes (numérique)

Ces données permettent de suivre l'évolution des polluants majeurs (PM2.5, PM10, NOx, SO2, etc.) dans le temps.

5.2 Données IHME : Global Burden of Disease

GBD 2023 (Global Burden of Disease) : Étude de l'Institute for Health Metrics and Evaluation (IHME) fournissant les taux de mortalité standardisés par âge pour les maladies respiratoires.

Les fichiers CSV contiennent environ 30 000 lignes et décrivent les taux normalisés d'incidence et de mortalité pour 100 000 habitants. Les variables incluent :

- L'année (numérique)
- Le pays (catégoriel)
- Le sexe (catégoriel)
- Les taux d'incidence et de décès (numériques)

Ces deux jeux de données sont complémentaires et permettent une analyse spatio-temporelle robuste. La littérature scientifique, notamment les travaux de Santé Publique France, confirme l'existence d'un lien entre pollution atmosphérique et santé respiratoire, justifiant pleinement leur mise en relation.

6 Périmètre Analytique

Le périmètre analytique du projet est défini comme suit :

TABLE 1 – Périmètre du projet

Dimension	Périmètre
Couverture géographique	Ensemble des pays du monde (197 pays)
Période	1980–2022
Polluants	PM2.5, PM10, NOx, SO2, CO, NH3, NMVOC, BC, OC
Maladies	Asthme, BPCO, cancer du poumon, pneumoconioses, maladies pulmonaires interstitielles

Hors périmètre : Les analyses à l'échelle intra-urbaine, ou celles portant sur les déterminants sociaux individuels, ne sont pas incluses en raison du manque de données disponibles.

7 Parties Prenantes

7.1 Commanditaire principal

L'**Agence Régionale de Santé (ARS)** constitue une partie prenante centrale dans ce projet, à la fois comme utilisatrice directe des analyses produites et comme actrice opérationnelle des décisions qui pourront en découler. En tant qu'autorité sanitaire de proximité, l'ARS est chargée de mettre en œuvre au niveau régional les politiques nationales liées à la prévention, à la surveillance épidémiologique et à la protection de la santé des populations.

Dans le contexte de la pollution atmosphérique, l'ARS joue un rôle d'interface entre les données scientifiques, les acteurs locaux et les décisions publiques. Les résultats du projet pourront être mobilisés pour ajuster les plans régionaux santé-environnement, cibler les territoires les plus vulnérables et renforcer les dispositifs de prévention.

7.2 Équipe projet

L'équipe projet est composée d'analystes et de spécialistes des données chargés de transformer les données brutes (EDGAR, IHME) en indicateurs intelligibles.

7.3 Acteurs secondaires

Les résultats du projet sont également destinés à des acteurs secondaires :

- **Santé Publique France** : intégration des indicateurs dans la surveillance sanitaire nationale
- **Observatoires régionaux de la qualité de l'air (ATMO)** : relais de l'information à l'échelle locale
- **Collectivités territoriales** (métropoles, régions) : déploiement des actions concrètes de réduction des émissions
- **Organismes de recherche** spécialisés en climat, pollution ou santé publique

8 Risques et Hypothèses

8.1 Risques identifiés

Plusieurs risques concernent la qualité et la comparabilité des données :

- Les méthodes de mesure diffèrent selon les pays
- Certaines séries peuvent être incomplètes
- Des facteurs socio-économiques non observés peuvent introduire des biais
- La granularité géographique est limitée à l'échelle des pays

8.2 Hypothèses

Les hypothèses principales du projet reposent sur :

- La fiabilité des données EDGAR et IHME
- La comparabilité des observations dans le temps et entre pays
- L'existence d'un lien statistique entre pollution atmosphérique et maladies respiratoires

Deuxième partie

Data Understanding

1 Indicateurs clefs

Dans le cadre de la méthodologie CRISP-DM, cette section présente les indicateurs clefs et les chaînes de valorisation identifiées pour l'exploration des données sur la pollution atmosphérique et les maladies respiratoires.

TABLE 2 – Indicateurs identifiés pour l'étude

Indicateur	Description	Unité
Émissions totales par polluant	Somme des émissions annuelles par type de polluant	Kilotonnes (kt)
Taux de mortalité standardisé	Décès pour 100 000 habitants ajusté par âge	Taux / 100 000 hab.
Corrélation polluant-maladie	Coefficient de Pearson entre émissions et mortalité	[-1, 1]
Part sectorielle	Pourcentage des émissions par secteur d'activité	%
Tendance temporelle	Variation annuelle moyenne des émissions/mortalité	% / an

2 Description des Données

2.1 Vue d'ensemble (TG12 – Préparation)

TABLE 3 – Caractéristiques des jeux de données

Caractéristique	EDGAR	GBD 2023
Nombre de pays	215	204
Période	1970–2022	1980–2023
Variables	9 polluants	5 maladies
Granularité	Pays, année, secteur	Pays, année, sexe
Unité	Kilotonnes (kt)	Taux pour 100 000 hab.

2.2 Polluants étudiés (EDGAR)

1. **PM2.5** : Particules fines ($< 2.5\mu m$)
2. **PM10** : Particules ($< 10\mu m$)
3. **NOx** : Oxydes d'azote
4. **SO2** : Dioxyde de soufre
5. **CO** : Monoxyde de carbone
6. **NH3** : Ammoniac
7. **NMVOC** : Composés organiques volatils non méthaniques
8. **BC** : Carbone noir
9. **OC** : Carbone organique

2.3 Maladies respiratoires (GBD)

1. Cancer de la trachée, des bronches et des poumons
2. Bronchopneumopathie chronique obstructive (BPCO) ou Maladie pulmonaire obstructive chronique dans le dataset
3. Asthme
4. Pneumoconioses
5. Maladies pulmonaires interstitielles

3 Qualité des Données

3.1 Valeurs manquantes (TG12 – Préparation)

L'analyse des valeurs manquantes révèle une excellente complétude des données. Le taux de valeurs manquantes global est inférieur à 0.01%, principalement concentré dans les correspondances de noms de pays entre les deux sources.

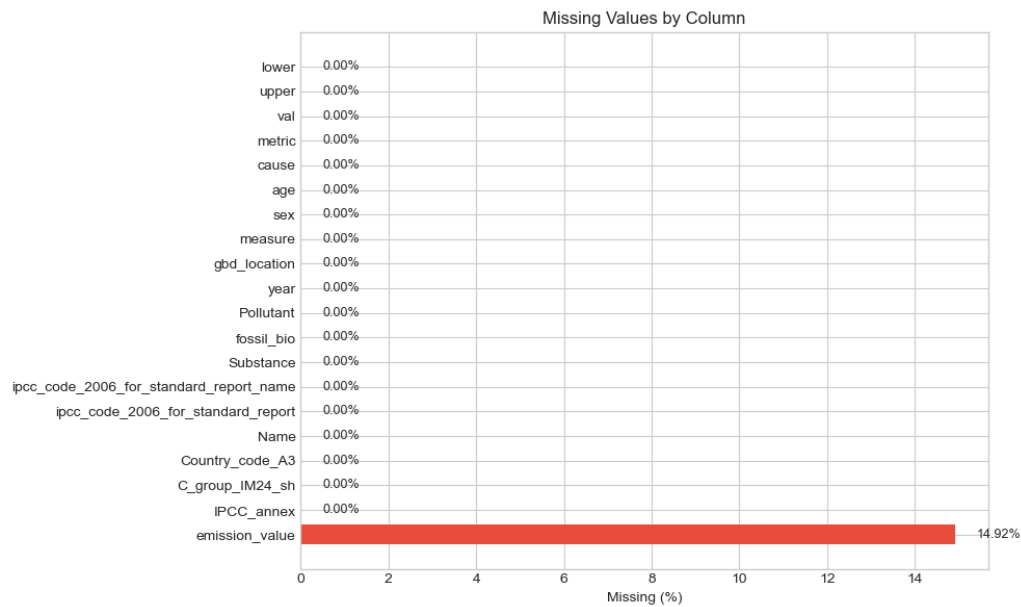


FIGURE 1 – Distribution des valeurs manquantes par variable

3.2 Jointure des données

La jointure des deux sources de données (EDGAR et GBD) a été réalisée sur :

- Le code ISO à 3 lettres des pays (197 pays communs)
- L'année (période commune : 1980–2022)

Le dataset final contient **38 millions d'observations** après jointure complète, et **63 445 observations** après agrégation par pays-année.

4 Analyse Exploratoire

4.1 Distribution des émissions par polluant (TG13 – Traitement)

La figure 2 montre la distribution des émissions pour chaque polluant. On observe une forte asymétrie positive (skewness) pour tous les polluants, avec quelques pays émettant des quantités significativement supérieures à la moyenne mondiale.

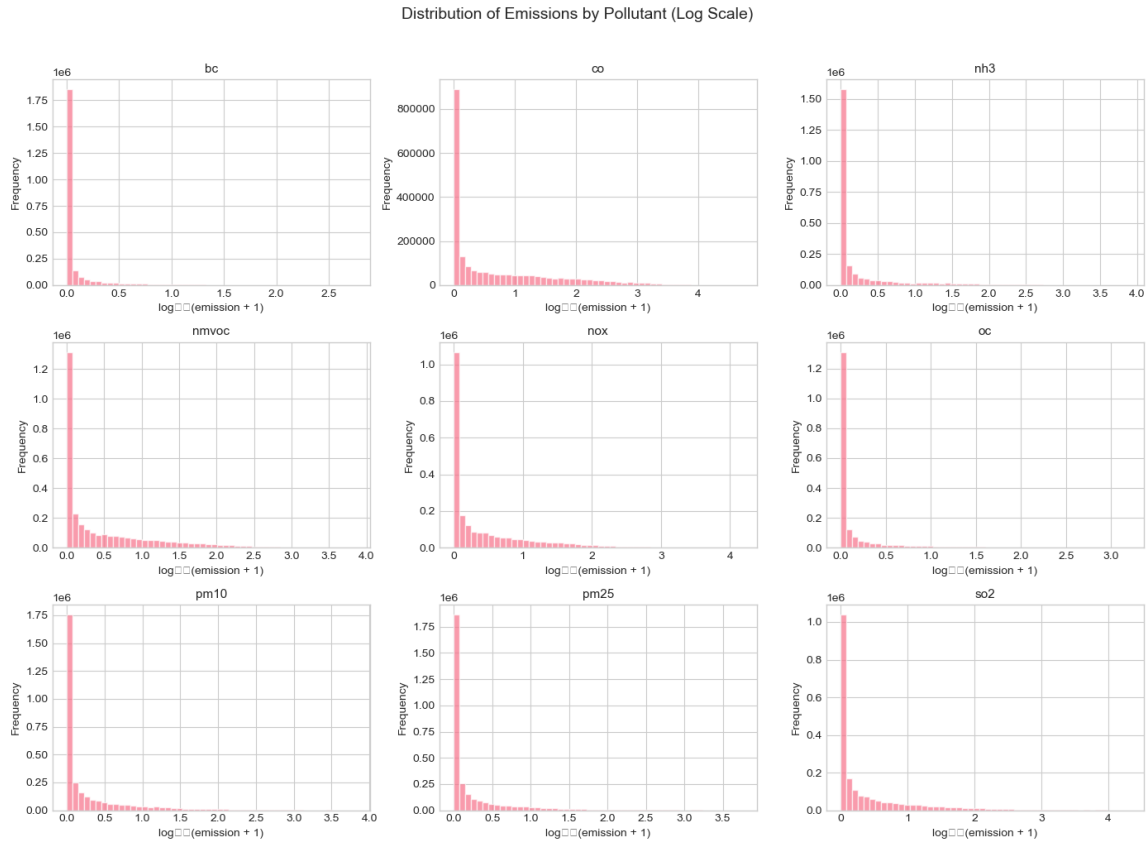


FIGURE 2 – Distribution des émissions par polluant (échelle logarithmique)

4.2 Comparaison hommes/femmes

L'analyse par sexe révèle des différences significatives dans les taux de mortalité. Les hommes présentent des taux plus élevés pour la majorité des maladies respiratoires, notamment pour le cancer du poumon et les BPCO.

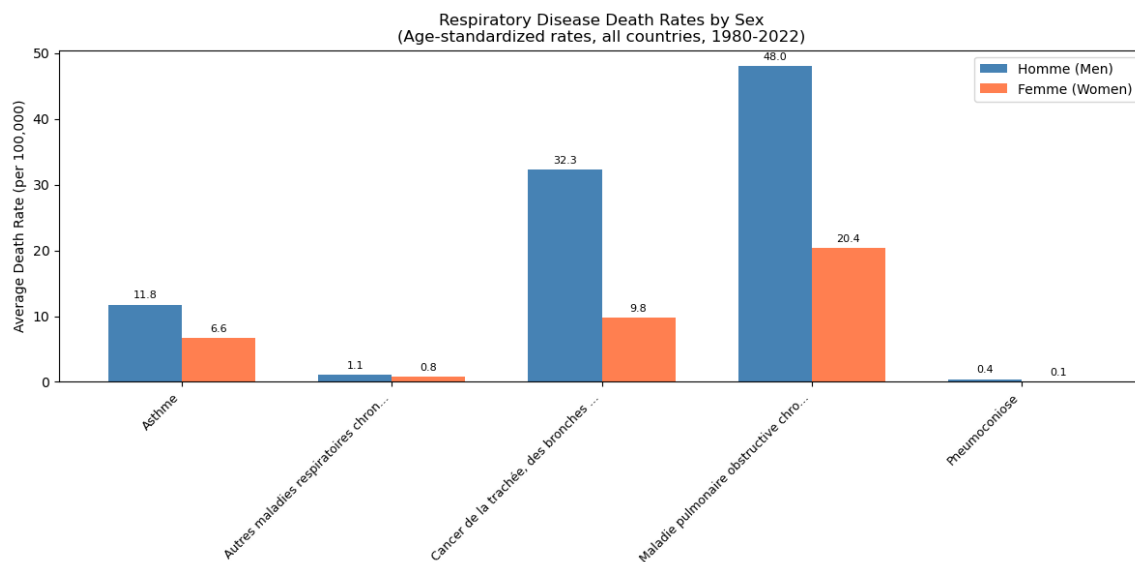


FIGURE 3 – Comparaison des taux de mortalité par sexe

5 Évolutions Temporelles

5.1 Tendances des émissions 1980–2022 (TG23 – Traitement)

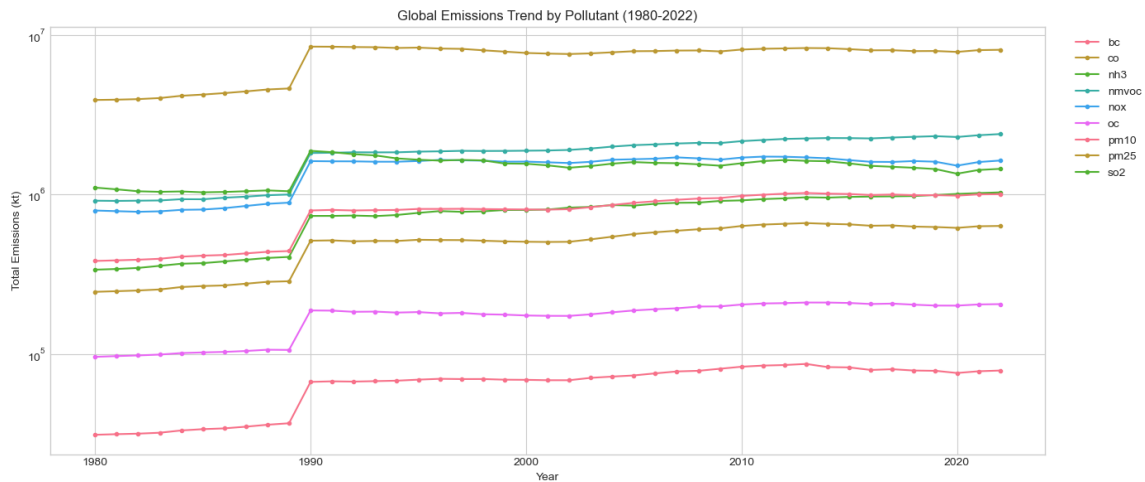


FIGURE 4 – Évolution temporelle des émissions mondiales par polluant

Observations clés :

- Pics d'émissions en 1990 pour tous les polluants
- Tous les polluants semblent très corrélés
- L'Oxide de Carbone (CO) est le polluant le plus présent

5.2 Tendances des décès 1980–2022 (TG23 – Traitement)

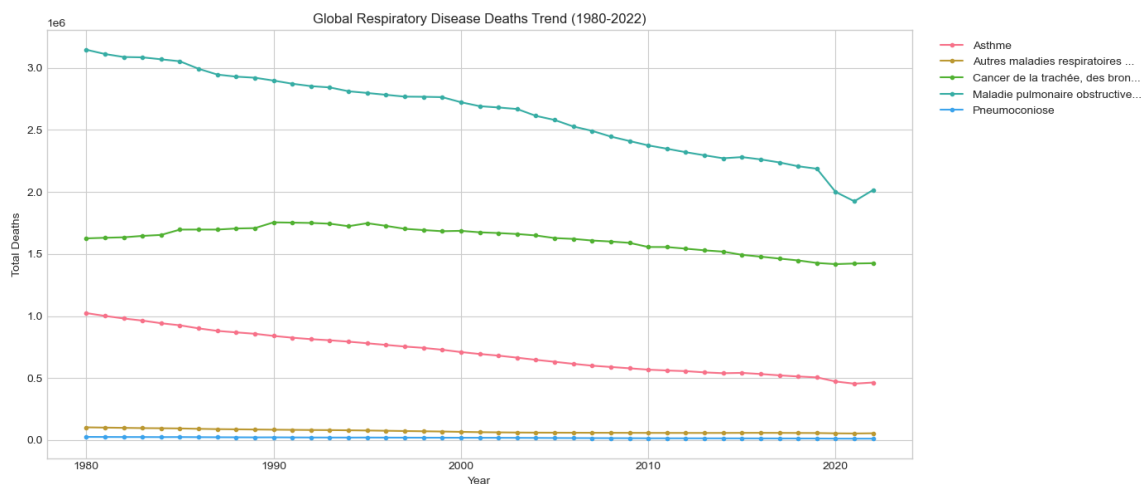


FIGURE 5 – Évolution temporelle des taux de mortalité par maladie

Observations clés :

- Diminution globale des taux de mortalité pour les BPCO
- Stabilité relative du cancer du poumon
- Baisse significative de l'asthme mortel

6 Analyse Géographique

6.1 Pays les plus émetteurs (TG24 – Restitution)

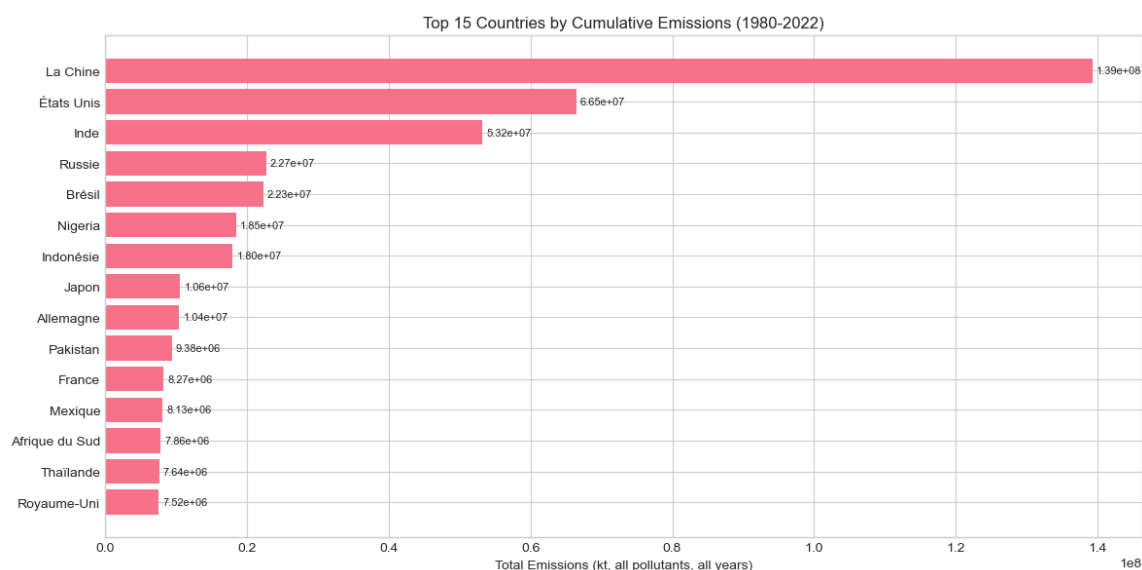


FIGURE 6 – Top 15 des pays émetteurs (toutes émissions confondues)

Les plus grands émetteurs sont la Chine, les États-Unis, l'Inde et la Russie, reflétant leur activité industrielle et leur population.

6.2 Pays avec les taux de mortalité les plus élevés

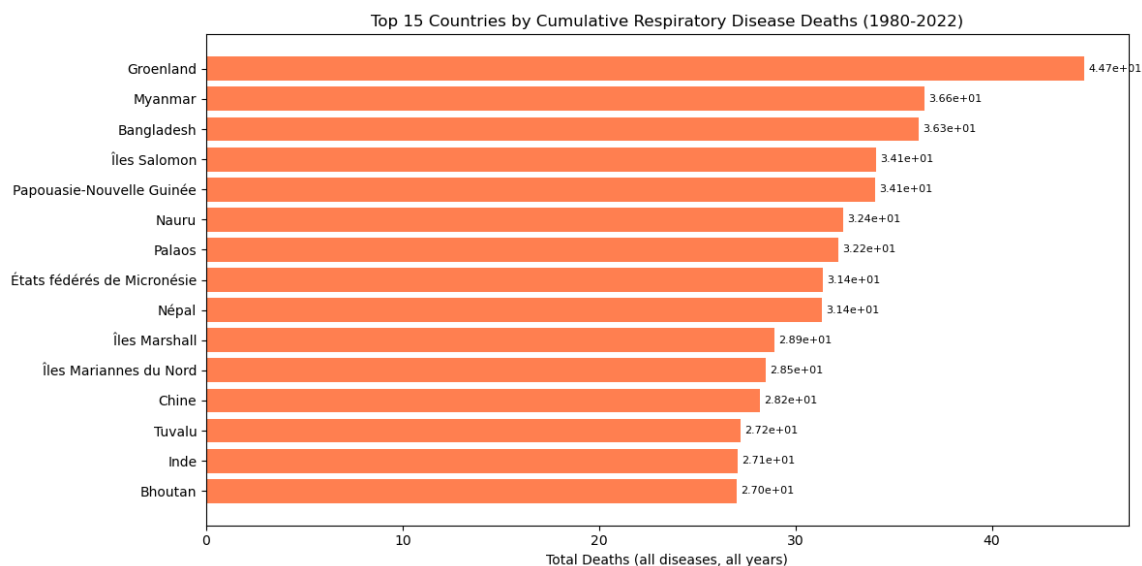


FIGURE 7 – Top 15 des pays par taux de mortalité respiratoire

6.3 Cartographie des maladies respiratoires (TG24 – Restitution)

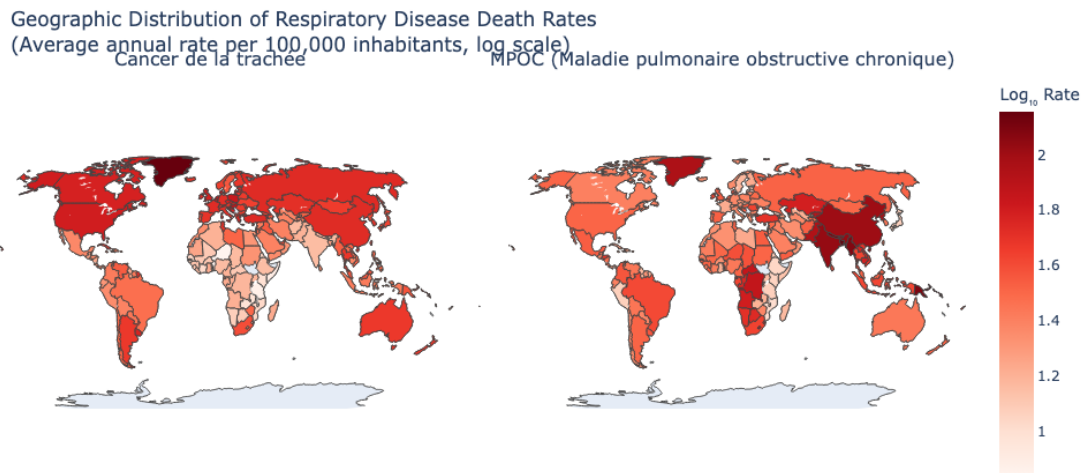


FIGURE 8 – Distribution géographique des taux de mortalité (cancer de la trachée et BPCO)

Observations :

- Le cancer de la trachée présente des taux élevés en Europe de l'Est, Amérique du Nord et Océanie
- La BPCO touche particulièrement l'Asie du Sud-Est (Chine, Inde) et certains pays africains
- Les pays développés montrent des patterns différents selon la maladie

6.4 Cartographie des émissions de polluants

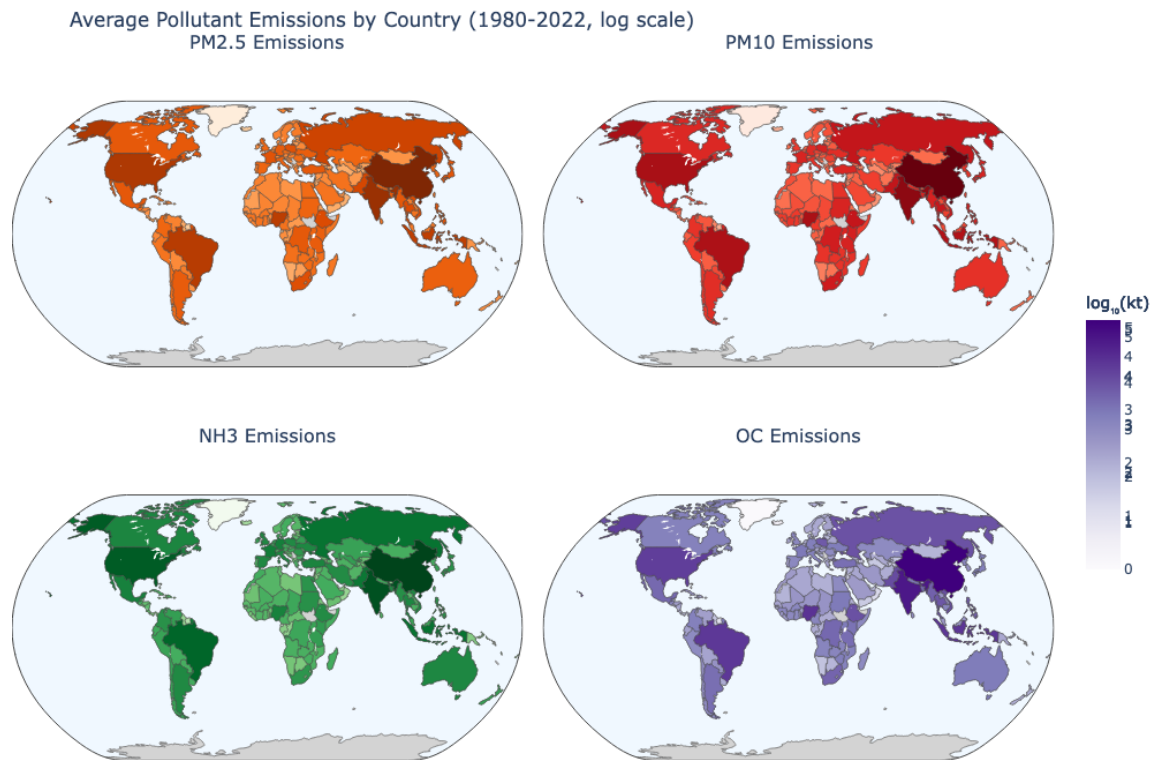


FIGURE 9 – Distribution géographique des émissions (PM2.5, PM10, NH3, OC)

Observations :

- La Chine et les États-Unis dominent pour tous les polluants
- Les particules fines (PM2.5, PM10) montrent des répartitions similaires
- L'ammoniac (NH3) est particulièrement élevé dans les pays à forte activité agricole (Inde, Chine, Brésil, États-Unis, Russie)
- Le carbone organique (OC) suit une distribution proche de celle des particules fines

7 Analyse Sectorielle

7.1 Secteurs d'activité (TG33 – Traitement)

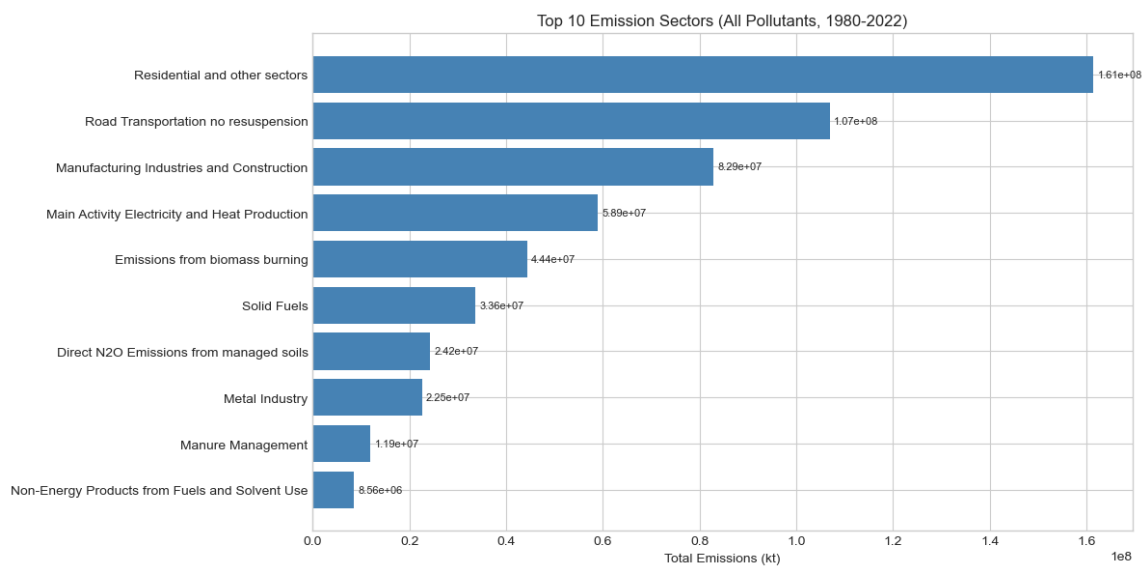


FIGURE 10 – Top 10 des secteurs d'activité par émissions totales

Les secteurs dominants sont :

- Transport routier
- Industrie manufacturière
- Production d'énergie
- Agriculture

7.2 Relation secteur-polluant (TG34 – Restitution)

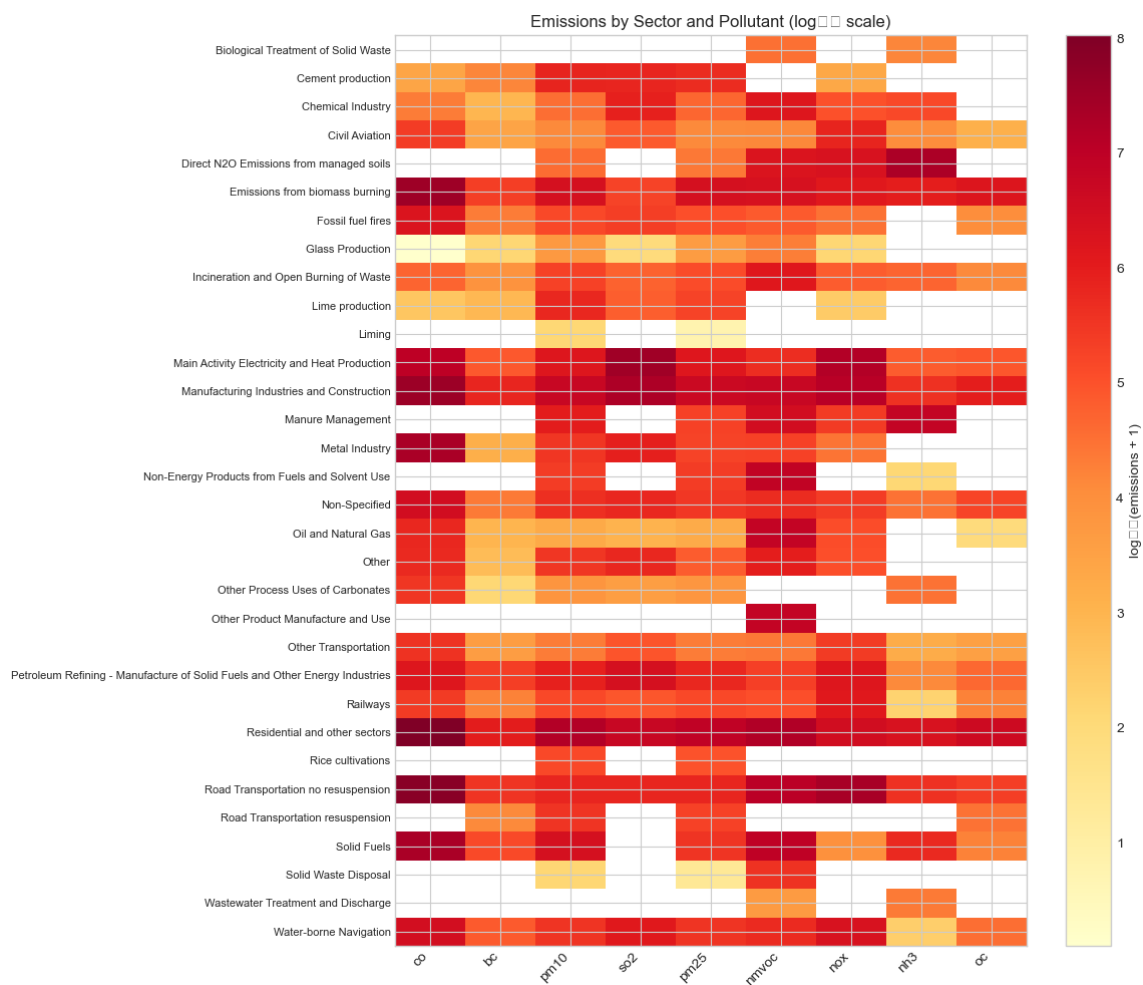


FIGURE 11 – Heatmap des émissions par secteur et polluant

Cette heatmap révèle les associations secteur-polluant : le transport routier est associé aux NOx et CO, tandis que l'agriculture domine les émissions de NH3.

8 Analyse des Corrélations

8.1 Corrélations polluants-maladies (TG13 – Traitement)

La matrice de corrélation entre polluants et maladies respiratoires constitue le cœur de cette étude exploratoire.

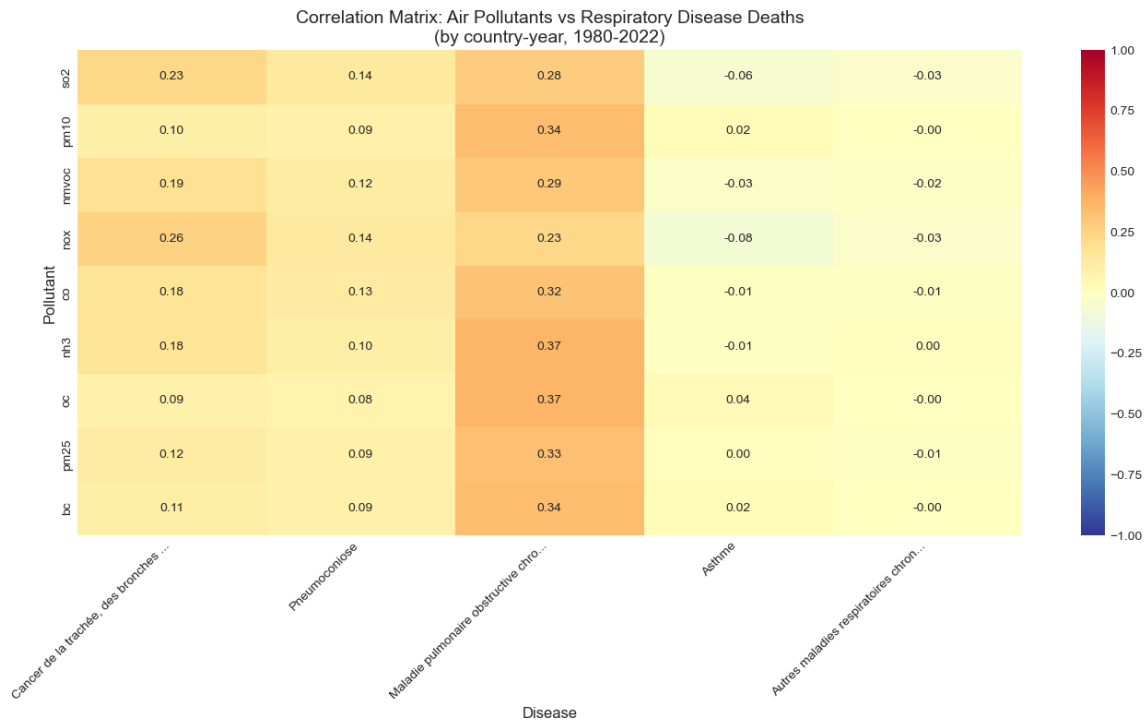


FIGURE 12 – Matrice de corrélation : polluants vs maladies respiratoires

Corrélations notables :

- PM2.5 et PM10 montrent des corrélations positives modérées avec toutes les maladies
- NH3 et OC présentent les corrélations les plus fortes avec le cancer du poumon
- les BPCO est corrélée à la plupart des polluants

8.2 Matrice de corrélation complète (TG14 – Restitution)

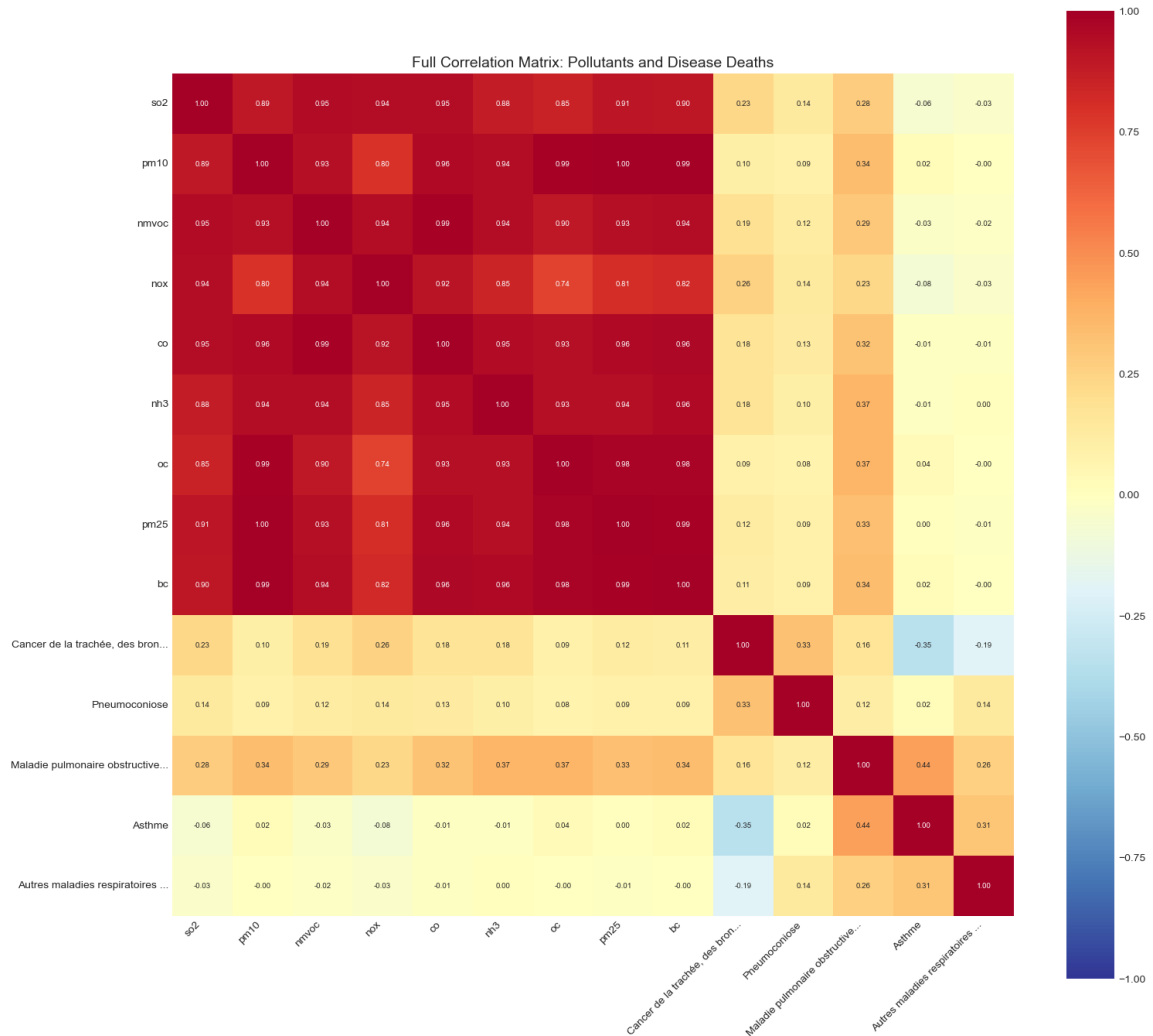


FIGURE 13 – Matrice de corrélation complète (polluants et maladies)

Observations :

- Forte multicollinéarité entre polluants ($r > 0.8$ pour la plupart)
- Les maladies sont également corrélées entre elles
- Ces corrélations suggèrent des facteurs communs (développement économique, urbanisation)
- les BPCO semblent plus corrélées aux polluants que les autres maladies (notamment NM-VOC, OC, PM10 et PM25)

8.3 Relations détaillées – scatter plots (TG14 – Restitution)



FIGURE 14 – Scatter plots : polluants clés vs maladies (échelle log)

9 Synthèse de l'Analyse Exploratoire

9.1 Qualité des données

TABLE 4 – Résumé de la qualité des données

Critère	Évaluation
Complétude	Excellente ($> 99.99\%$)
Couverture géographique	197 pays
Couverture temporelle	43 ans (1980–2022)
Cohérence des unités	Vérifiée
Outliers	$< 2\%$ (à traiter)

9.2 Principales conclusions

1. **Corrélations significatives** : Des corrélations positives modérées existent entre les polluants atmosphériques et les maladies respiratoires, particulièrement pour les particules fines (PM2.5, PM10), les NVMOC, l'Oxide de Carbone et le cancer du poumon ainsi que les bronchopneumopathies chroniques obstructives (BPCO) .
2. **Multicolinéarité** : Les polluants sont fortement corrélés entre eux, ce qui nécessitera une attention particulière lors de la modélisation (régularisation, réduction de dimension).
3. **Disparités géographiques** : Les pays en développement présentent des taux d'émission croissants tandis que les pays développés montrent des tendances à la baisse.
4. **Différences par sexe** : Les hommes sont plus touchés par les maladies respiratoires, avec des taux de mortalité 2 à 3 fois supérieurs pour le cancer du poumon.
5. **Évolution temporelle** : Malgré l'augmentation des émissions globales, les taux de mortalité standardisés tendent à diminuer, suggérant l'impact positif des avancées médicales.

9.3 Recommandations pour la modélisation

- Appliquer une transformation logarithmique aux émissions
- Considérer une analyse en composantes principales (ACP) pour réduire la multicolinéarité
- Inclure des variables de contrôle (PIB, urbanisation, accès aux soins)
- Utiliser des modèles de panel pour exploiter la dimension temporelle
- Tester des modèles avec décalage temporel (lag) entre exposition et maladie

10 Chaînes de valorisation

Trois chaînes de valorisation principales ont été identifiées pour cette phase d'exploration.

10.0.1 Chaîne 1 – Analyse des corrélations polluants-maladies

Objectif : Identifier les liens statistiques entre les émissions de polluants atmosphériques et les taux de mortalité par maladie respiratoire.

TABLE 5 – Chaîne 1 – Corrélations polluants-maladies

Tâche (TG)	générique	Tâches spécifiques (TS)	Outils
TG11 – Collecte et Intégration		Télécharger les données EDGAR (émissions) et GBD (mortalité), extraire les fichiers Excel/CSV	Python (requests), Excel, CSV
TG12 – Préparation et Mise en qualité		Harmoniser les noms de pays (ISO 3166), traiter les valeurs manquantes, uniformiser les unités	Polars, Pandas
TG13 – Traitement et Analyse		Calculer les matrices de corrélation, identifier les associations significatives	Python (scipy, numpy), Omniscopie
TG14 – Restitution et Visualisation		Générer les heatmaps de corrélation, scatter plots polluant vs maladie	Matplotlib, Seaborn, Omniscopie

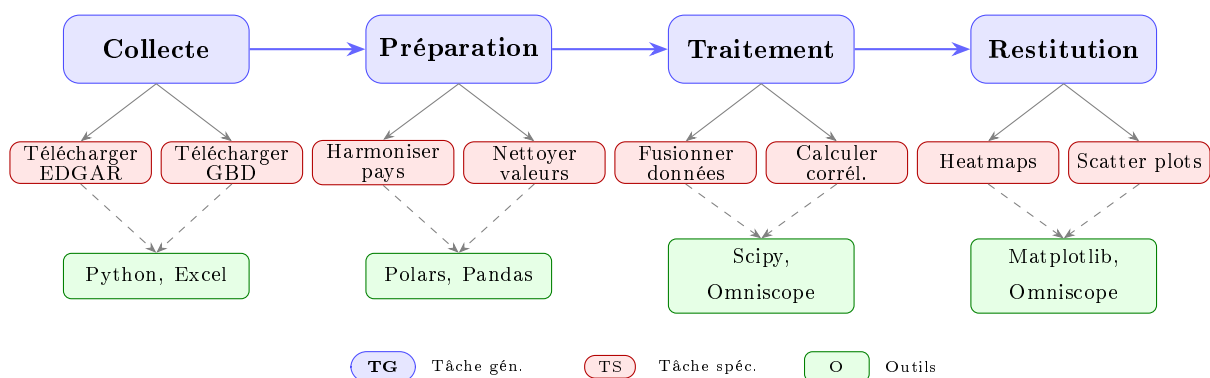


FIGURE 15 – Schéma de la chaîne 1 – Corrélations polluants-maladies

10.0.2 Chaîne 2 – Analyse géographique et temporelle

Objectif : Comprendre la distribution spatiale et l'évolution temporelle des émissions et de la mortalité respiratoire.

TABLE 6 – Chaîne 2 – Analyse géographique et temporelle

Tâche générique (TG)	Tâches spécifiques (TS)	Outils
TG21 – Collecte et Intégration	Joindre les données EDGAR et GBD sur code ISO pays et année	Polars (join)
TG22 – Préparation et Mise en qualité	Agréger par pays-année, filtrer la période commune (1980-2022), 197 pays	Polars (group_by, filter)
TG23 – Traitement et Analyse	Calculer les classements (top émetteurs, top mortalité), tendances temporelles	Python (numpy, pandas), Omnisciope
TG24 – Restitution et Visualisation	Graphiques bar charts par pays, courbes d'évolution temporelle	Matplotlib, Seaborn, Omnisciope

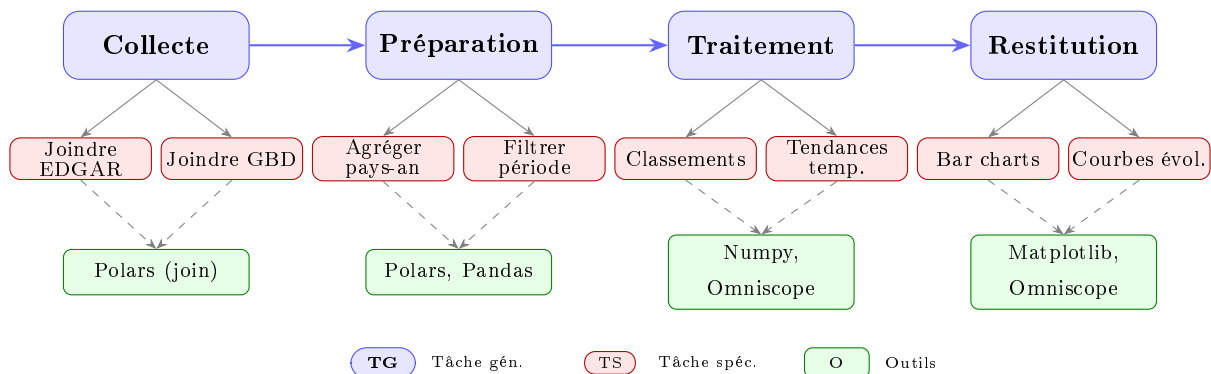


FIGURE 16 – Schéma de la chaîne 2 – Analyse géographique et temporelle

10.0.3 Chaîne 3 – Analyse sectorielle des émissions

Objectif : Identifier les secteurs d'activité les plus polluants pour orienter les analyses et recommandations.

TABLE 7 – Chaîne 3 – Analyse sectorielle

Tâche générique (TG)	Tâches spécifiques (TS)	Outils
TG31 – Collecte et Intégration	Extraire les données EDGAR avec granularité sectorielle (IPCC categories)	Python, Excel
TG32 – Préparation et Mise en qualité	Regrouper les secteurs par catégorie IPCC, normaliser les noms	Polars, Pandas, Omniscope
TG33 – Traitement et Analyse	Calculer les émissions totales par secteur et polluant, identifier les associations secteur-polluant	Python (numpy), Omniscope
TG34 – Restitution et Visualisation	Heatmap secteur-polluant, bar charts des top secteurs	Matplotlib, Seaborn, Omniscope

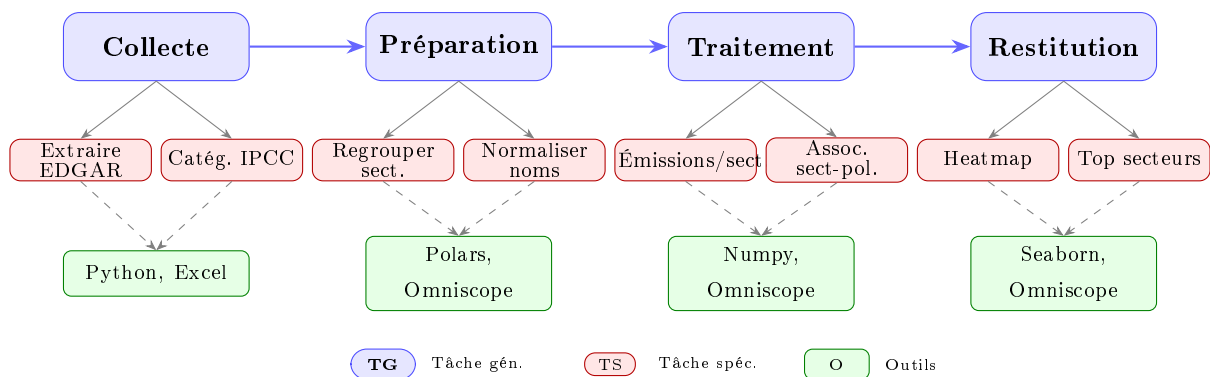


FIGURE 17 – Schéma de la chaîne 3 – Analyse sectorielle des émissions

Troisième partie

Data Preparation

1 Jeux de données (rappel)

- **EDGAR** : <https://edgar.jrc.ec.europa.eu/>
- **GBD 2023** : <https://ghdx.healthdata.org/gbd-2023>

Pour plus de précisions, voir la section [Jeux de Données](#).

2 Extraction des données

Les données étant statiques et n'ayant pas pour but d'être récoltées périodiquement, elles ont simplement été téléchargées à la main depuis les sources respectives. Pour les données du dataset *EDGAR*, il a suffi de télécharger les différents fichiers Excel correspondant à chaque polluant sur le site de la Commission Européenne. Pour les données du dataset *GBD*, nous avons dû remplir plusieurs formulaires par paquets en prenant soin de sélectionner les mêmes critères de mesures afin de ne pas excéder les quotas, la réponse à notre demande d'extension des dits quotas étant arrivée assez tardivement.

3 Prétraitement des données

3.1 Fusion des données (TG41-43 – Transformation)

La partie la plus considérable de l'étape de préparation des données s'est portée sur la création d'un dataset unique et cohérent à partir des données extraites. Ce travail a été effectué avant l'exploration des données : en effet, il est nécessaire de fusionner les données des différents datasets pour pouvoir les exploiter ensemble. Nous avons utilisé la bibliothèque *Polars* de Python pour effectuer cette fusion. En effet, malgré notre plus grande maîtrise de la bibliothèque *Pandas*, nous avons choisi d'utiliser *Polars* en raison de son efficacité et de sa capacité à gérer plus rapidement et efficacement des données relativement volumineuses. Cet outil, écrit en Rust, offre des performances bien supérieures et une syntaxe similaire à celle de *Pandas* (bien que parfois plus complexe), ce qui facilite la transition entre les deux bibliothèques. Dans les cas où nos connaissances de *Pandas* pouvaient permettre des opérations plus rapides (e.g. : avec *matplotlib* ou *seaborn*), une simple conversion avec `df.to_pandas()` suffit.

Ainsi, *polars* a été utilisé pour effectuer les opérations suivantes :

```
1 import polars as pl
2
3 edgar_dirs = [d for d in Path('data/EDGAR').iterdir() if d.is_dir()]
4
5 poll_dfs = {}
6
7 for subdir in edgar_dirs:
8     df = pl.read_excel(
9         list(subdir.glob('*.xlsx'))[0], engine='calamine',
10         read_options={"header_row": 9}
11     )
12     poll_dfs[subdir.name.lower()] = df
13
14 edgar = pl.concat(
15     [df.with_columns(pl.lit(name).alias("Pollutant")) for name, df in poll_dfs.items()]
16     ,
17     how="vertical"
```

Code 1 – Extraction des données EDGAR avec Polars

L'extraction des données GBD se fait de manière similaire. Un challenge a été les noms de pays : afin de fusionner les données EDGAR et GBD, nous devions joindre sur les années et les pays. Or, si EDGAR nous fournit des noms de pays en anglais et des codes ISO 3166-1 alpha-3, soit une catégorisation propre, GBD lui nous fournit des noms de pays en français mal traduit. En effet, même en naviguant le site en anglais avec un navigateur en anglais, le dataset GBD renvoie des noms de pays en français avec une traduction automatique. Ce qui donne des apostrophes mal encodées, des articles de manière aléatoire (e.g. : France, Espagne, mais "*La Chine*"). Cela nous a forcé à créer une table de traduction des noms de pays en français vers ceux en anglais, à la main, dans un dictionnaire `country_mapping` exporté dans un fichier CSV.

Une fois cela fait, il ne restait plus qu'à dépivoter les années du dataset EDGAR (qui étaient des colonnes au format Y_NNNN) et à fusionner les datasets, ce qui nous donne un énorme dataset qui regroupe sur une ligne pour chaque pays, chaque année, chaque polluant, chaque maladie et chaque industrie les émissions du polluant et les morts causés par la maladie. Ce dataset brut est intéressant pour une analyse préliminaire, mais il est nécessaire d'ensuite grouper les données simplement par pays, années et polluant pour obtenir un dataset plus exploitable.

```

1 emissions_agg = (
2     combined2
3     .group_by(['gbd_location', 'Country_code_A3', 'year', 'Pollutant'])
4     .agg(pl.col('emission_value').sum().alias('total_emission'))
5     .pivot(on='Pollutant', index=['gbd_location', 'Country_code_A3', 'year'], values='
      total_emission')
6 )
7
8 ...
9
10 correlation_data = emissions_agg.join(
11     deaths_agg,
12     on=['gbd_location', 'Country_code_A3', 'year'],
13     how='inner'
14 )
15
16 correlation_data.write_parquet('data/combined/correlation_ready.parquet')

```

Code 2 – Agrégation du dataset combiné pour la corrélation

Joined data for correlation : (8471, 17)
 Countries : 197
 Years : 43
 Countries with ISO codes : 197
 shape : (5, 17)

C'est à partir de ce dataset que nous avons obtenu notre matrice de corrélation et c'est à partir de celui-ci que nous avons commencé à développer notre modèle.

3.2 Stockage des données (TG44 – Export)

Le dataset brut étant assez lourd, pour optimiser le stockage, nous avons utilisé le format *Parquet* afin de pouvoir facilement stocker, charger et manipuler les données. Cela permettait en outre de rendre plus facile le partage de ces fichiers, évitant des fichiers CSV de plusieurs gigaoctets contre seulement quelques mégaoctets.

Le format *Parquet*, un format binaire ouvert, est particulièrement adapté à notre utilisation de Polars, étant donné qu'il est supporté nativement et peut donc être lu et transformé en dataframe d'un simple appel à la méthode `read_parquet`. Sa structure columnaire stocke les données par colonnes plutôt que par lignes, ce qui est idéal pour les opérations analytiques où seules certaines colonnes sont nécessaires, réduisant ainsi les entrées-sorties. Contrairement aux fichiers texte comme le CSV, le caractère binaire de Parquet permet une compression avancée et une lecture plus rapide.

4 Préparation pour la modélisation

Une fois le dataset fusionné et stocké au format Parquet, les données doivent être préparées pour la modélisation prédictive. Cette étape comprend le chargement des données, l'application de transformations, la gestion des valeurs manquantes, la normalisation des features et la séparation train/test.

4.1 Décalage temporel (Lag)

Un point clef de la préparation pour la modélisation est l'introduction d'un décalage temporel entre les émissions (variables explicatives) et les décès (variable cible). En effet, il existe un délai biologique entre l'exposition à la pollution atmosphérique et le développement de maladies respiratoires. Dans notre cas, les données sont préparées de telle sorte que les émissions de l'année t prédisent les décès de l'année $t + lag$, où `lag` est le nombre d'années considérées.

Dans les faits, le décalage temporel a été testé et n'a pas permis d'améliorer les performances du modèle. Nous avons donc décidé de garder la fonctionnalité, mais de ne pas l'utiliser en choisissant un lag égal à zéro. Les résultats des tests de lag sont présentés ci-dessous :

```

1 LAG ANALYSIS BY DISEASE
2 =====
3
4 Random State: 1
5
6 Lung Cancer:
7   Lag 0 years | R2 = 0.8567
8   Lag 3 years | R2 = 0.8625
9   Lag 6 years | R2 = 0.8720
10  Lag 9 years | R2 = 0.8660
11
12 Pneumoconiosis:
13   Lag 0 years | R2 = 0.8058
14   Lag 3 years | R2 = 0.7868
15   Lag 6 years | R2 = 0.7367
16   Lag 9 years | R2 = 0.7928
17
18 COPD:
19   Lag 0 years | R2 = 0.7937
20   Lag 3 years | R2 = 0.7811
21   Lag 6 years | R2 = 0.7924
22   Lag 9 years | R2 = 0.8119
23
24 ...

```

Code 3 – Analyse des décalages temporels par maladie

Comme montré dans la sortie de script ci-dessus, les performances varient légèrement selon le décalage temporel choisi, mais aucun pattern consistant n'émerge. Un décalage de 0 années fournit un bon équilibre et reste notre choix par défaut.

4.2 Gestion des valeurs manquantes (TG51 – Préparation ML)

Le dataset combiné contient quelques valeurs manquantes dûes aux enregistrements manquants pour les émissions de polluants dans certains pays certaines années.

Nous avons opté pour la stratégie la plus simple et la plus robuste : **supprimer les lignes contenant des valeurs manquantes**. Cette approche, bien qu'elle réduise la taille du dataset, garantit la qualité des données utilisées pour le modèle et évite les artefacts introduits par l'imputation. L'imputation remplace les valeurs manquantes par une estimation statistique fixe calculée sur l'ensemble de la colonne, comme la moyenne, la médiane ou la valeur la plus fréquente. L'interpolation est une autre méthode possible, elle consiste à deviner la valeur manquante en observant la tendance des valeurs voisines (juste avant et juste après), souvent de manière linéaire. Elle est particulièrement adaptée aux séries temporelles, mais peut introduire des artefacts si les données sont bruitées ou si la tendance n'est pas linéaire.

4.3 Transformations des features (TG51 – Préparation ML)

4.3.1 Transformation logarithmique

Les émissions de polluants présentent souvent une distribution asymétrique avec des valeurs aberrantes. Pour normaliser cette distribution et réduire l'influence des valeurs extrêmes, nous appliquons une transformation logarithmique :

$$x'_i = \log(1 + x_i)$$

où x_i est l'émission du polluant i . La constante additive 1 ($\log1p$) permet de gérer les valeurs zéro ou très proches de zéro sans générer d'erreurs numériques.

4.3.2 Normalisation des features

Après la transformation logarithmique, nous normalisons et réduisons toutes les features à l'aide d'un `StandardScaler` du framework de machine-learning *scikit-learn*. Cette normalisation ramène les features à une moyenne de 0 et un écart-type de 1 :

$$x'_i = \frac{x_i - \mu}{\sigma}$$

où μ et σ sont la moyenne et l'écart-type estimés sur l'ensemble d'entraînement. Cette étape est essentielle pour les modèles sensibles à l'échelle (régression linéaire, réseaux de neurones, etc.).

4.4 Sélection des features et cibles

Pour la modélisation, nous utilisons les neuf polluants comme features explicatives :

- CO (Monoxyde de carbone)
- SO₂ (Dioxyde de soufre)
- NO_x (Oxydes d'azote)
- NMVOC (Composés organiques volatils non méthaniques)
- PM_{2.5} (Particules fines)
- PM₁₀ (Particules grossières)
- NH₃ (Ammoniac)
- BC (Carbone noir)
- OC (Carbone organique)

Pour les cibles (variables à prédire), nous avons sélectionné trois maladies respiratoires principales basées sur l'analyse exploratoire :

- **Lung Cancer** : Cancer de la trachée, des bronches et des poumons
- **Pneumoconiosis** : Pneumoconiose
- **COPD** : Bronchopneumopathie chronique obstructive

Les deux autres maladies (Asthme et Autres maladies respiratoires chroniques) ont été exclues car l'analyse exploratoire n'a révélé aucune corrélation significative avec les polluants étudiés.

4.5 Séparation train/test

Le dataset est divisé en deux ensembles :

- **Ensemble d'entraînement** (80%) : utilisé pour ajuster les paramètres du modèle et calibrer le `StandardScaler`
- **Ensemble de test** (20%) : utilisé pour évaluer les performances du modèle sur des données non vues pendant l'entraînement

Cette séparation en 80/20 est assez standard et permet une évaluation fiable des performances du modèle dans la grande majorité des cas.

4.6 Implémentations

Deux versions de la pipeline de préparation ont été implémentées dans le module `model.data_prep` :

- `prepare_data()` : pour scikit-learn, retournant un objet `PreparedData` avec les arrays NumPy
- `prepare_data_torch()` : pour PyTorch, retournant un objet `PreparedDataTorch` avec les tensors PyTorch

Les deux versions partagent la même logique de préparation et de normalisation, garantissant la cohérence entre les deux frameworks.

4.7 Analyse des scenarii

Le module inclut également des fonctions qui nous seront utiles dans la partie modélisation pour l'analyse de scenarii post-modélisation (`get_baseline_emissions()`, `create_scenario_features()`), permettant d'évaluer l'impact de réductions d'émissions sur la mortalité prédite. Cela facilite l'interprétabilité du modèle et le support à la décision.

5 Chaîne de valorisation

Une chaîne de valorisation a été identifiée pour la phase de préparation des données, regroupant l'extraction, le nettoyage, la fusion et le stockage.

5.0.1 Chaîne 4 – Préparation et intégration des données

Objectif : Construire un dataset unifié, cohérent et optimisé à partir des sources EDGAR et GBD pour permettre les analyses exploratoires et la modélisation.

TABLE 8 – Chaîne 4 – Préparation et intégration des données

Tâche générique (TG)	Tâches spécifiques (TS)	Outils
TG41 – Extraction des données brutes	Télécharger les fichiers Excel EDGAR (par polluant), télécharger les CSV GBD (par paquets, respect des quotas), charger les fichiers avec Polars	Python, Polars (read_excel, read_csv), calamine
TG42 – Nettoyage et Harmonisation	Créer le mapping des noms de pays FR→EN, harmoniser les codes ISO 3166-1 alpha-3, traiter les valeurs manquantes et les apostrophes mal encodées	Polars, dictionnaire Python, CSV
TG43 – Transformation et Fusion	Dépivoter les colonnes années EDGAR (Y_NNNN), joindre EDGAR et GBD sur pays/année, agréger par pays/année/polluant pour la corrélation	Polars (unpivot, join, group_by, pivot)
TG44 – Stockage et Export	Exporter les datasets en format Parquet (compression, performance), vérifier l'intégrité des données	Polars (write_parquet)

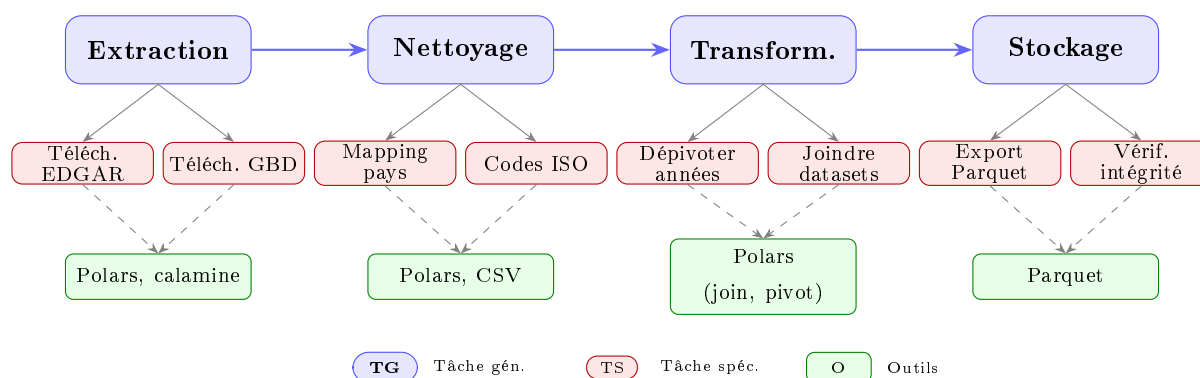


FIGURE 18 – Schéma de la chaîne 4 – Préparation et intégration des données

6 Synthèse de la préparation des données

La préparation des données est une étape cruciale dans le processus qui doit aboutir à la modélisation. Une donnée mal préparée peut complètement ruiner un modèle. Il est donc capital de passer par les étapes de compréhension business, compréhension des données et de préparation des données, dans cet ordre. Bien qu'une partie de ce processus s'effectue dans le cadre de la compréhension des données, il faut noter que les objectifs sont différents : c'est pourquoi nous avons deux jeux de données : un pour l'exploration plus globale (plus fourni car encore à défricher) et un pour des observations plus fines et pour la modélisation (plus compact car déjà pré-traité). Le dataset utilisé pour la modélisation est le suivant :

```
1
2 Saved aggregated dataset: 8,471 rows ×17 columns
3 File: data/combined/correlation_ready.parquet
4
5 Columns: ['gbd_location', 'Country_code_A3', 'year', 'pm10', 'oc', 'nmvoc', 'nox', '
  pm25', 'nh3', 'co', 'so2', 'bc', 'Asthme', 'Cancer de la trachée, des bronches et
  des poumons', 'Pneumoconiose', 'Autres maladies respiratoires chroniques', '
  Maladie pulmonaire obstructive chronique']
```

Code 4 – Sortie du script de préparation des données et présentation du dataset final

Quatrième partie

Modelization

1 Réflexions préliminaires

1.1 Modèles, machine-learning et deep-learning

L'étape du modèle du framework CRISP-DM implique la réflexion autour de modèles mathématiques permettant d'exprimer des relations entre nos données, relations qui peuvent s'avérer complexes et difficiles à identifier à l'œil nu. Ces modèles plus ou moins complexes, peuvent notamment se baser sur des méthodes statistiques ou des approches d'apprentissage automatique. Ces modèles sont stochastiques, c'est-à-dire non-déterministes. Ainsi, deux exécutions du même modèle avec les mêmes données mais des initialisations différentes peuvent produire des résultats différents, d'où l'importance d'une graine aléatoire (`random_state`) fixée pour assurer la reproductibilité.

L'apprentissage automatique (*machine learning*) regroupe des algorithmes qui apprennent automatiquement à partir des données sans être explicitement programmés. Ces algorithmes cherchent à identifier des patterns dans les données pour faire des prédictions sur de nouvelles données. On distingue plusieurs catégories : la régression (prédire une valeur continue), la classification (prédire une catégorie), le clustering (grouper les données), etc. Des exemples courants incluent la régression linéaire, les arbres de décision, les forêts aléatoires et les machines à vecteurs de support (SVM). Ces modèles sont généralement interprétables et relativement rapides à entraîner.

L'apprentissage profond (*deep learning*) est un sous-ensemble du machine learning basé sur les réseaux de neurones artificiels (artificial neural networks). Contrairement aux modèles classiques, les réseaux de neurones apprennent des représentations hiérarchiques des données, où chaque couche du réseau transforme les données en abstractions de plus en plus complexes. Bien que les réseaux de neurones soient plus exigeants en termes de données et de puissance de calcul, ils peuvent capturer des relations non-linéaires très complexes et sont particulièrement efficaces pour les données de haute dimension.

Dans notre contexte de prédiction de mortalité due à la pollution, nous explorons à la fois des modèles classiques de machine learning (pour l'interprétabilité et la performance) et des approches de deep learning (pour la flexibilité et la capacité à modéliser des relations complexes potentiellement cachées dans les données). Nous avons de prime abord choisi cette approche à des fins éducatives : à la fois dans le but de pratiquer ces différentes techniques et d'identifier leurs différences une fois mis en application, mais également pour démontrer leurs forces et leurs faiblesses.

1.2 Technologies choisies

Afin de mettre en œuvre un algorithme de machine learning, nous nous sommes dirigés vers la bibliothèque *scikit-learn*. *sklearn*, de son nom abrégé, est une bibliothèque Python open-source qui implémente de nombreux algorithmes de machine learning. C'est le standard dans l'industrie et du côté de la recherche pour le machine learning dit "classique", de la préparation des données à la modélisation. Scikit-learn et les algorithmes de machine learning qu'il implémente sont particulièrement adaptés aux problèmes de régression sur des données tabulaires comme le notre. *PyTorch*, quant à elle, est une bibliothèque Python open-source développée par Meta pour le deep learning.

PyTorch n'est a priori pas l'outil idéal pour traiter un problème de régression simple comme le notre : il peut fonctionner, mais l'intérêt des réseaux de neurones se situe dans leur capacité à représenter les données d'une manière beaucoup plus complexe, par exemple en utilisant des couches de convolution pour traiter des images. Ayant déjà travaillé avec ces deux bibliothèques, nous savons donc que PyTorch introduit une complexité supplémentaire qui n'est a priori pas nécessaire pour notre problème. Cependant, toujours pour des raisons éducatives, nous avons décidé de traiter les deux, avec un réseau de neurones basique pour PyTorch et un modèle d'arbres de décision pour scikit-learn. Nous aurons donc deux modèles, et en réalité plus que cela en raison des différentes variables cibles.

1.3 Choix des variables

Dans un premier temps, les variables d'entrées, les features, étaient tous les polluants + les années. Après analyse, nous avons décidé d'exclure la variable temporelle (année) de l'ensemble d'entraînement. En effet, l'inclusion de l'année pourrait conduire le modèle à apprendre une simple tendance temporelle (tendance à la baisse de la mortalité due aux progrès médicaux) plutôt que la relation causale entre pollution et santé. Notre objectif étant d'isoler l'impact des polluants, nous avons privilégié une approche agnostique du temps. De même, nous avons décidé de ne pas garder l'Asthme et les autres maladies respiratoires, étant donné que la corrélation avec les variables d'entrées était autour de 0 voire légèrement en dessous. Nous nous sommes donc concentrés sur les bronchopneumopathies chroniques obstructives (BPCO), et dans une moindre mesure sur les cancers de la trachée et des poumons, ainsi que sur les pneumoconioses, qui avaient une corrélation positive avec les variables d'entrées.

1.4 Méthodes d'évaluation d'un modèle

Il existe une multitude de méthodes pour évaluer de prime abord les performances d'un modèle. Les deux que nous verrons ici sont l'écart quadratique moyen (RMSE, pour *Root Mean Squared Error*) et le coefficient de détermination (R^2). Le R^2 mesure la proportion de variance des données expliquée par le modèle : un R^2 de 1 indique un modèle parfait, tandis qu'un R^2 de 0 signifie que le modèle ne fait pas mieux que prédire la moyenne.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2} \quad (1)$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2} = 1 - \frac{SS_{res}}{SS_{tot}} \quad (2)$$

$\hat{y}_i$: la prédiction du modèle pour la donnée i

y_i : la valeur réelle de la donnée i

$\bar{y}$: la moyenne des valeurs réelles

2 Implémentation itérative des modèles

2.1 Modèle de machine learning scikit-learn (TG52 – Modélisation classique)

Dans un premier temps, le temps de calcul et l'écriture du code étant très rapide avec scikit-learn une fois nos données prétraitées et nos features/targets définies, nous avons décidé de comparer plusieurs algorithmes de machine learning pour trouver le meilleur modèle. Nous avons utilisé les algorithmes suivants :

- Ridge (régression linéaire)
- Forêts aléatoires
- Boost de Gradients

2.1.1 Ridge

Ridge est une extension de la régression linéaire classique. Dans la régression linéaire standard, on cherche à trouver une ligne (ou un hyperplan en haute dimension) qui minimise l'erreur entre les prédictions et les valeurs réelles. Cependant, lorsqu'on a de nombreuses features ou que les données sont bruitées, la régression linéaire peut *overfitter* (sur-apprendre) en mémorisant le bruit des données d'entraînement au lieu de généraliser à de nouvelles données.

Ridge résout ce problème en ajoutant une pénalité (*regularization term*) à la fonction d'erreur. Cette pénalité décourage les coefficients du modèle de devenir trop grands. Intuitivement, cela force le modèle à être plus « conservateur » : au lieu de chercher à ajuster parfaitement chaque point de données, il accepte une légère erreur mais conserve des coefficients modérés et stables. Le paramètre α contrôle la force de cette pénalité : une valeur faible permet aux coefficients d'être grands (plus proche de la régression linéaire classique), tandis qu'une valeur élevée les force à être petits.

2.1.2 Forêts aléatoires (Random Forests)

Les forêts aléatoires sont un ensemble (*ensemble learning*) d'arbres de décision. Un arbre de décision fonctionne en posant une série de questions sur les features (par exemple, « $\text{PM}_{2.5} > 10$? ») et en suivant les branches selon les réponses jusqu'à atteindre une prédiction. Bien que faciles à comprendre, un seul arbre a tendance à overfitter en créant trop de branches spécialisées.

Une forêt aléatoire construit des centaines d'arbres de décision, chacun entraîné sur un sous-ensemble aléatoire des données (avec remise) et utilisant des subsets aléatoires des features. Chaque arbre fait une prédiction, et la prédiction finale est la **moyenne** (pour la régression) ou le **vote majoritaire** (pour la classification) de tous les arbres. Cette approche d'ensemble réduit drastiquement l'overfitting car les erreurs d'apprentissage des arbres individuels se compensent mutuellement. De plus, les forêts aléatoires peuvent capturer des relations non-linéaires et des interactions entre features sans nécessiter d'ingénierie manuelle des features.

2.1.3 Gradient Boosting

Le Gradient Boosting (ou Boost de Gradients) fonctionne également en construisant un ensemble d'arbres, mais de manière séquentielle et intelligente. Contrairement aux forêts aléatoires où tous les arbres sont indépendants, le Gradient Boosting entraîne les arbres un par un. Chaque nouvel arbre est entraîné pour **corriger les erreurs** du modèle précédent en se concentrant sur les cas mal prédits.

Concrètement, le premier arbre fait une prédiction imparfaite. Le deuxième arbre apprend à prédire les « résidus » (les erreurs) du premier arbre. Le troisième arbre apprend les résidus du premier plus le deuxième, et ainsi de suite. À la fin, la prédiction finale est la somme des contributions de tous les arbres. Cette approche itérative permet au modèle d'améliorer graduellement ses prédictions, similaire à un processus d'apprentissage où on apprend de ses erreurs. Le Gradient Boosting est réputé pour sa haute performance sur de nombreuses tâches et peut capturer des relations très complexes.

2.1.4 Modèle retenu

Il n'est pas nécessaire à notre niveau de comprendre en détails comment fonctionne chaque modèle d'un point de vue théorique pour obtenir un résultat satisfaisant, au delà de ce qui a été évoqué précédemment. En revanche, il est tout de même nécessaire de faire un choix éclairé. Nous avons donc comparé ces modèles avec notre pipeline de préparation et une boucle très simple :

```

1 results = {}
2 for name, model in models.items():
3     # Train
4     model.fit(data.X_train, data.y_train)
5     # Predict
6     y_pred = model.predict(data.X_test)
7     # Evaluate
8     r2 = r2_score(data.y_test, y_pred)
9     rmse = np.sqrt(mean_squared_error(data.y_test, y_pred))
10
11 results[name] = {'model': model, 'r2': r2, 'rmse': rmse}
12 print(f"{name:20s} | R² = {r2:.4f} | RMSE = {rmse:.1f}")

```

Code 5 – Comparaison des modèles scikit-learn

Cette boucle nous donne la sortie suivante :

```

1 Ridge | R² = 0.1564 | RMSE = 9188.9
2 Random Forest | R² = 0.7373 | RMSE = 5128.3
3 Gradient Boosting | R² = 0.7715 | RMSE = 4782.2

```

Code 6 – Comparaison des modèles scikit-learn (sortie de script)

Le modèle qui se démarque ici est celui du boost de gradient, qui présente un R^2 de 0.7715 et un RMSE de 4782.2. C'est donc le modèle que nous avons choisi pour notre analyse. On notera que le code scikit-learn est extrêmement simple et va droit au but. `model.fit()` pour entraîner le modèle et `model.predict()` pour obtenir les prédictions. Une fois que l'on a bien récupéré nos données, c'est tout ce qu'il faut pour obtenir des prédictions.

2.2 Modèle de deep learning PyTorch (TG53 – Modélisation deep learning)

2.2.1 Réseaux de neurones artificiels (ANN)

Les réseaux de neurones artificiels (ANN) sont des modèles de machine learning organisés en couches. Ils sont constitués d'une entrée, d'une ou plusieurs couches cachées (*hidden layers*) et d'une couche de sortie. Chaque couche est composée de neurones qui sont connectés entre eux par des poids et des biais déterminés lors de l'entraînement. Les neurones sont activés par une fonction d'activation qui permet de moduler la sortie du neurone en fonction de sa somme pondérée d'entrée. Les neurones combinent linéairement leurs entrées ($z = w \cdot x + b$) avant d'appliquer une fonction d'activation qui introduit de la non-linéarité.

Les fonctions d'activation sont essentielles pour permettre aux réseaux de neurones de capturer des relations non-linéaires. Sans elles, même un réseau multicouche ne serait qu'une combinaison linéaire inutile. Pour les couches cachées, nous utilisons :

- **Linéaire** : définie par $f(z) = z$. Elle ne transforme pas l'entrée et est utilisée dans la couche de sortie pour les problèmes de régression (comme notre prédiction de mortalité), où nous voulons une sortie continue non bornée. Bien qu'il n'y ait pas de transformation non-linéaire, la couche de sortie possède toujours des **poids** et des **biais** qui sont appris durant l'entraînement. Cette couche combine linéairement les activations de la couche cachée précédente selon $y = W \cdot h + b$, où W est la matrice des poids et b est le vecteur de biais
- **ReLU (Rectified Linear Unit)** : définie par $f(z) = \max(0, z)$. Elle retourne 0 si l'entrée est négative, sinon retourne l'entrée telle quelle. C'est la fonction la plus populaire pour les couches cachées car elle est simple, rapide à calculer, et encourage la parcimonie des activations.

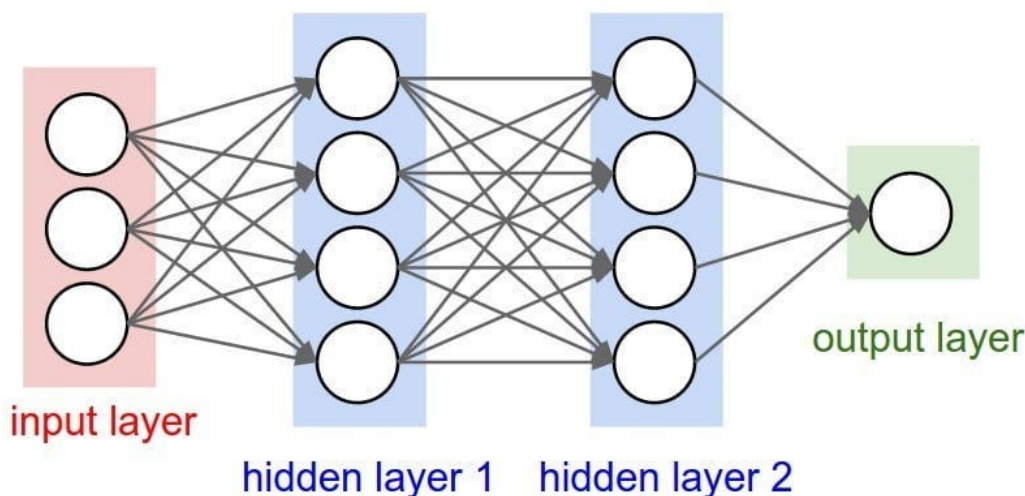


FIGURE 19 – Artificial Neural Network (ANN)

Le processus d'apprentissage comporte deux phases : la **propagation avant** (*forward pass*) et la **rétropropagation** (*backpropagation*).

Lors de la propagation avant, une donnée d'entrée traverse le réseau couche par couche. Chaque neurone reçoit les activations de la couche précédente, les multiplie par ses poids, ajoute son biais, puis applique sa fonction d'activation. Cette sortie devient l'entrée des neurones de la couche suivante, jusqu'à atteindre la couche de sortie qui produit la prédiction finale.

Ensuite, on compare la prédiction avec la vraie valeur pour calculer l'erreur (loss). La rétropropagation calcule comment cette erreur dépend de chaque poids et biais du réseau en utilisant la dérivation en chaîne. Ces gradients indiquent dans quelle direction chaque poids doit être ajusté pour réduire l'erreur.

L'entraînement suit un processus itératif :

1. **Propagation avant** : passer les données à travers le réseau pour obtenir une prédiction
2. **Calcul de l'erreur** : comparer la prédiction avec la vraie valeur (ex. : erreur quadratique moyenne)
3. **Rétropropagation** : calculer les gradients via la dérivation en chaîne
4. **Mise à jour des poids** : ajuster les poids et biais dans la direction opposée au gradient (descente de gradient) avec un pas d'apprentissage (*learning rate*) qui contrôle la taille des ajustements

Ce cycle est répété sur plusieurs *epochs* (passes complètes sur les données) jusqu'à convergence du modèle, c'est-à-dire quand l'erreur se stabilise. Un learning rate trop élevé peut faire *diverger* le modèle (la loss explose au lieu de diminuer), tandis qu'un learning rate trop faible ralentit la convergence. Des problèmes d'*overfitting* peuvent survenir lorsque le modèle est trop complexe ou entraîné trop longtemps : il mémorise le bruit des données d'entraînement au lieu de généraliser. Ce principe s'applique à tous les modèles de machine learning. C'est pour résoudre cela qu'il faut ajuster les hyperparamètres (learning rate, epochs, régularisation, etc.).

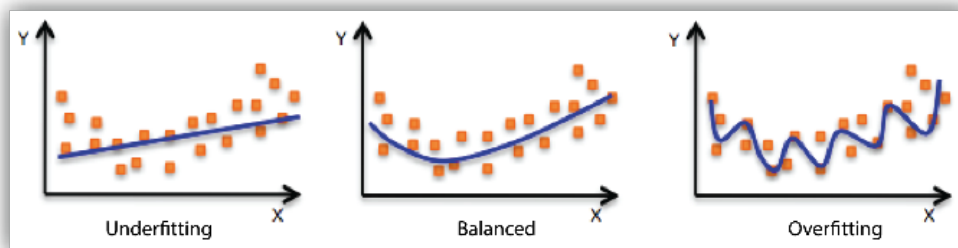


FIGURE 20 – Overfitting et Underfitting : illustration des deux extrêmes

2.2.2 Notre modèle d'ANN

```

1 class MLP(nn.Module):
2     def __init__(self, n_features):
3         super().__init__()
4         self.layers = nn.Sequential(
5             nn.Linear(n_features, 64),
6             nn.ReLU(),
7             nn.Linear(64, 32),
8             nn.ReLU(),
9             nn.Linear(32, 1)
10        )
11
12    def forward(self, x):
13        return self.layers(x)

```

Code 7 – Notre ANN

C'est le modèle le plus basique que nous puissions définir. Il est composé de deux couches cachées avec une fonction d'activation ReLU entre chaque couche. C'est tout. En effet, nous ne traitons pas de données catégorielles et notre dataset n'est pas très complexe ou volumineux : nous travaillons sur un problème de régression avec des données numériques tabulaires.

2.2.3 Boucle d'entraînement

Avec PyTorch, contrairement à scikit-learn, nous devons définir nous-mêmes la boucle d'entraînement. Voici un exemple de boucle d'entraînement pour notre modèle :

```
1 def train_model(data, epochs=500, lr=0.001): # Lower lr, more epochs
2     model = MLP(data.n_features)
3     optimizer = torch.optim.Adam(model.parameters(), lr=lr)
4     loss_fn = nn.MSELoss()
5
6     loader = DataLoader(
7         TensorDataset(data.X_train, data.y_train.unsqueeze(1)),
8         batch_size=64, shuffle=True
9     )
10
11     for epoch in range(epochs):
12         for X_batch, y_batch in loader:
13             optimizer.zero_grad()
14             loss = loss_fn(model(X_batch), y_batch)
15             loss.backward()
16             torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0) # Prevent
17                                     exploding gradients
18             optimizer.step()
19     return model
```

Code 8 – Boucle d'entraînement de notre modèle

Nous avons choisi un optimiseur Adam et l'erreur quadratique moyenne (MSE, pour *Mean Squared Error*) comme fonction de perte. Il ne nous reste plus qu'à appliquer notre boucle d'entraînement sur nos données et de regarder le score R^2 obtenu.

3 Implémentation finale et résultats

3.1 Implémentation

3.1.1 Implémentation PyTorch : ANN

L'implémentation PyTorch est celle présentée dans la section précédente. Pour rappel :

```

1 class MLP(nn.Module):
2     def __init__(self, n_features):
3         super().__init__()
4         self.layers = nn.Sequential(
5             nn.Linear(n_features, 64),
6             nn.ReLU(),
7             nn.Linear(64, 32),
8             nn.ReLU(),
9             nn.Linear(32, 1)
10        )
11
12    def forward(self, x):
13        return self.layers(x)

```

Code 9 – Notre ANN (rappel)

3.1.2 Implémentation scikit-learn : GradientBoostingRegressor

```

1 for disease in DISEASES:
2     from data_prep import FEATURES
3     data = prepare_data(target_disease=disease, lag_years=0, feature_names=FEATURES)
4
5     # fine tuned for COPD with gridsearchCV
6     model = GradientBoostingRegressor(n_estimators=300, max_depth=7, learning_rate=0.1,
7                                       random_state=20)
8     model.fit(data.X_train, data.y_train)
9
10    r2 = r2_score(data.y_test, model.predict(data.X_test))
11
12    models[disease] = {'model': model, 'r2': r2, 'data': data}
13
14    short_name = DISEASE_SHORT_NAMES[disease]
15    print(f"{short_name:25s} | R2 = {r2:.4f}")

```

Code 10 – Modèle scikit-learn final

3.1.3 Problèmes rencontrés

Nous avons dû adapter notre modèle pour contrer l'overfitting présent dans notre configuration initiale des hyperparamètres. Nous avons utilisé GridSearchCV afin de tester plusieurs combinaisons et de déterminer les meilleurs hyperparamètres pour le modèle de régression gradient boosting, en nous concentrant en particulier sur les BPCO. Une fois cela fait, nous avons pu obtenir des résultats plus satisfaisants.

Toutefois, nous nous sommes heurtés à un problème assez classique en machine learning. Notre modèle est assez précis et il est très bon pour l'interpolation des données, c'est-à-dire pour prédire des valeurs dans une plage de valeurs connues. En revanche, les modèles basés sur des

arbres de décisions sont très mauvais en ce qui concerne la prédiction des valeurs hors de la plage de valeurs connues, c'est-à-dire l'extrapolation. Nous avons donc un modèle relativement simple et performant, mais limité pour l'étape suivante qui consistait à créer des scénarii.

3.2 Résultats (TG54 – Évaluation)

En effet, notre modèle de régression gradient boosting (GB) a obtenu un R^2 assez satisfaisant pour les données de test. Ce résultat est satisfaisant car il indique que notre modèle est capable de prédire correctement les valeurs dans une plage de valeurs connues.

```

1 GRADIENT BOOSTING - ALL DISEASES
2 =====
3 Lung Cancer | R2 = 0.9232
4 Pneumoconiosis | R2 = 0.8054
5 COPD | R2 = 0.9007
6
7 =====
8 TOP 5 POLLUTANTS PER DISEASE
9 =====
10 Lung Cancer | so2 (25%), nh3 (17%), nox (13%), oc (10%), co (9%)
11 Pneumoconiosis | oc (16%), nh3 (15%), nmvoc (13%), pm10 (12%), nox (12%)
12 COPD | nh3 (18%), oc (18%), nmvoc (15%), so2 (10%), nox (10%)

```

Code 11 – Résultats du modèle de régression gradient boosting

Nous pouvons également observer quelles features ont été les plus importantes pour notre modèle de régression GB. Cette information est capitale, c'est elle qui nous permet de comprendre quelles variables ont le plus d'influence sur la prédiction de la maladie respiratoire. Le modèle a identifié des liens entre des polluants spécifiques et les maladies respiratoires, sans nécessiter de connaissances préalables sur les mécanismes physiologiques sous-jacents.

En ce qui concerne notre modèle ANN, nous avons des résultats un peu moins probants à première vue :

```

1 PYTORCH MLP - ALL DISEASES
2 =====
3 Lung Cancer | R2 = 0.8718
4 Pneumoconiosis | R2 = 0.9558
5 COPD | R2 = 0.6374

```

Code 12 – Résultats du modèle ANN

En particulier, le modèle se retrouve en difficulté à prédire les bronchopneumopathies chroniques obstructives (COPD). Cela peut être attribué à la complexité des mécanismes physiologiques impliqués dans ces maladies et à la nécessité d'un modèle plus sophistiqué pour prendre en compte ces interactions.

3.3 Prédictions (TG61-64 – Simulation de scenarii)

Une fois les modèles entraînés et évalués, nous les avons utilisés pour effectuer des prédictions sur des scenarii hypothétiques de réduction des émissions de polluants. L'objectif est de simuler l'impact de politiques environnementales sur la mortalité due aux maladies respiratoires.

3.3.1 Analyse de scenarii

Nous avons défini plusieurs scenarii de réduction des émissions sur 10 ans par rapport aux niveaux actuels (année de référence : 2022) :

- **Niveau actuel** : 100% des émissions (baseline)
- **Réduction de 5%** : 95% des émissions
- **Réduction de 10%** : 90% des émissions
- **Réduction de 20%** : 80% des émissions

Pour chaque scénario, nous appliquons le multiplicateur correspondant aux émissions moyennes de l'année de référence, puis nous utilisons les modèles entraînés pour prédire le taux de mortalité associé à chaque maladie. Cela permet d'estimer les bénéfices potentiels en termes de santé publique d'une réduction des émissions polluantes.

3.3.2 Résultats des prédictions – Gradient Boosting

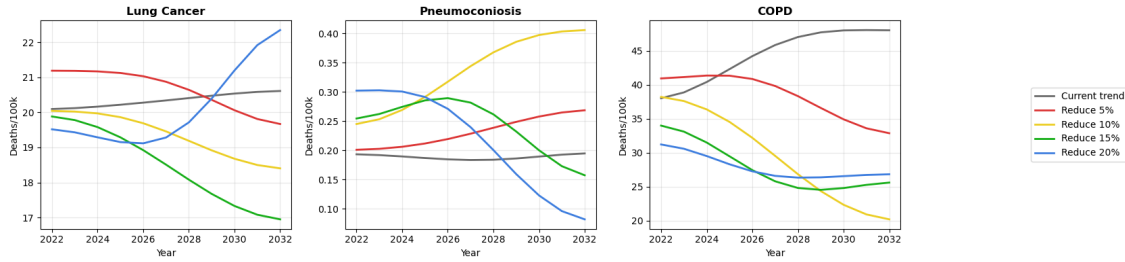


FIGURE 21 – Prédictions des taux de mortalité selon les scenarii de réduction – Gradient Boosting

Les prédictions du modèle Gradient Boosting présentent des incohérences notables. Pour le cancer du poulmon, une réduction de 20% des émissions entraîne paradoxalement une augmentation de la mortalité prédite, ce qui est contre-intuitif. Les prédictions pour la pneumoconiose et la BPCO sont également erratiques et ne suivent pas une tendance logique de diminution avec la réduction des émissions. Ces résultats mettent en évidence les limites du modèle Gradient Boosting pour ce type d'analyse prospective.

3.3.3 Résultats des prédictions – Réseau de neurones (ANN)

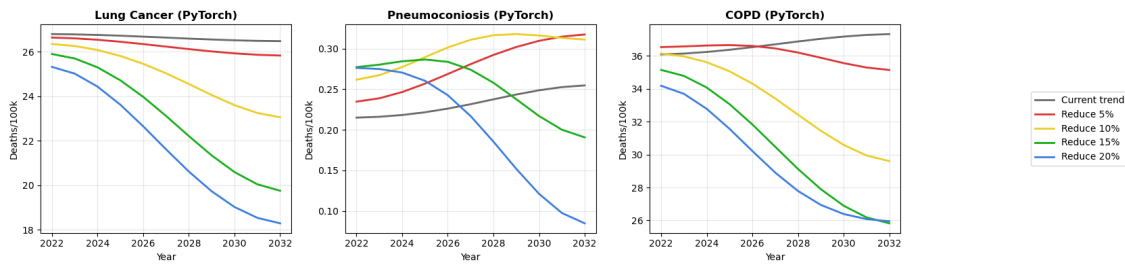


FIGURE 22 – Prédictions des taux de mortalité selon les scenarii de réduction – ANN

Les prédictions du réseau de neurones sont globalement plus cohérentes. On observe une tendance générale à la diminution de la mortalité avec la réduction des émissions, ce qui correspond à l'intuition médicale et épidémiologique. On note toutefois une légère anomalie pour la pneumoconiose, où les réductions de 5% et 10% produisent des prédictions légèrement supérieures au niveau actuel avant de diminuer pour des réductions plus importantes. Malgré cette irrégularité mineure, le modèle ANN capture mieux la relation attendue entre émissions et mortalité.

3.3.4 Interprétation et limites

Ces résultats illustrent plusieurs points importants :

- Les modèles de machine learning, même performants en interpolation (sur des données similaires au set d'entraînement), peuvent échouer en extrapolation (sur des données hors distribution). C'est particulièrement vrai pour les méthodes basées sur les arbres (Gradient Boosting) qui sont par nature limitées aux bornes vues lors de l'entraînement (fonction en escalier), contrairement aux réseaux de neurones qui approximent une fonction continue.
- Le réseau de neurones semble mieux généraliser la relation sous-jacente entre émissions et mortalité, offrant une "pente" de réduction plus cohérente.
- Les prédictions doivent être interprétées avec prudence : elles ne constituent pas des prévisions épidémiologiques précises, mais plutôt des indications de tendances potentielles.

- D'autres facteurs non modélisés (politiques de santé, évolution démographique, progrès médicaux) influencent également la mortalité.

4 Implémentation dans Omniscope

Dans le cadre de NF21, nous avons appris à utiliser **Omniscope**, outil de *Business Intelligence (BI)* permettant notamment de créer des tableaux de bord interactifs. C'est l'usage principal, que nous en avons fait au cours de nos séances de travaux pratiques, mais dans le cadre de ce projet, nous pouvons utiliser Omniscope pour implémenter des modèles statistiques. En effet, dans la partie permettant la préparation des données, il est possible d'ajouter des blocs de la catégorie *Analyse* afin d'appliquer des modèles de machine learning sur les données préparées.

4.1 Portage de la pipeline de préparation

La première étape consiste à répliquer notre pipeline de préparation des données dans Omniscope. Nous avons tout d'abord voulu utiliser le fichier Parquet généré par notre script Python, étant donné qu'Omniscope supporte nativement ce format de fichier. Cependant, il semble que notre fichier contienne des métadonnées qui ne sont pas supportées par Omniscope. Qu'à cela ne tienne, nous avons repris nos fichiers *edgar.csv* et *gbd.csv* et avons recréé la pipeline de préparation des données en utilisant les blocs natifs d'Omniscope. Nous avons pu gagner du temps en réutilisant ces fichiers déjà traités et en nous basant sur les processus déjà créés. Le résultat est le même que celui obtenu avec notre script Python, à savoir un dataset agrégé par pays et par année, avec les polluants et les maladies respiratoires.

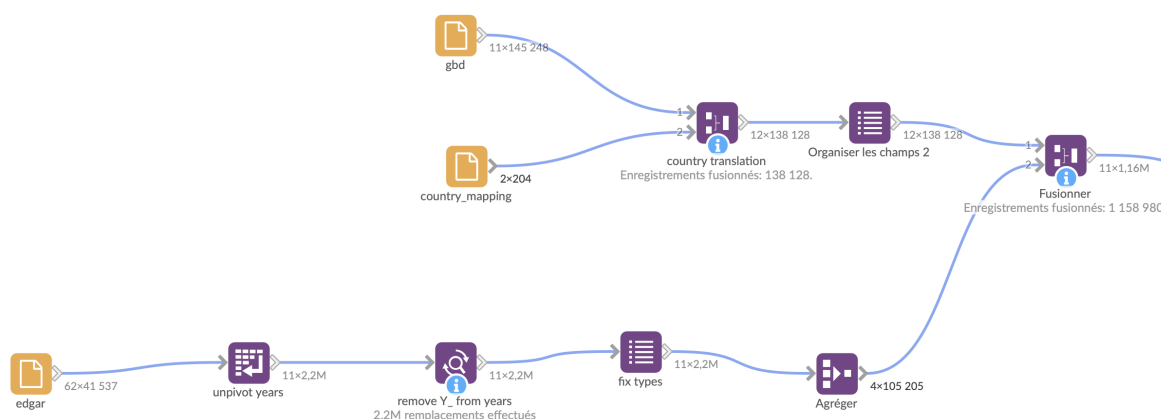


FIGURE 23 – Pipeline de préparation des données dans Omniscope

Nous avons chargé le dataset GBD ainsi qu'un fichier contenant les traductions de pays afin d'éviter les problèmes rencontrés plus tôt. Nous avons chargé le dataset EDGAR, dépivoté et renommé les années puis nous nous avons corrigé les types de données des deux datasets, agrégé les données par pays et par année puis avons fusionné les deux datasets sur les colonnes pays et année. Enfin, nous n'avons gardé que les décès comme métrique et les BPCO comme maladie cible.

4.2 Modèles de régression dans Omniscope

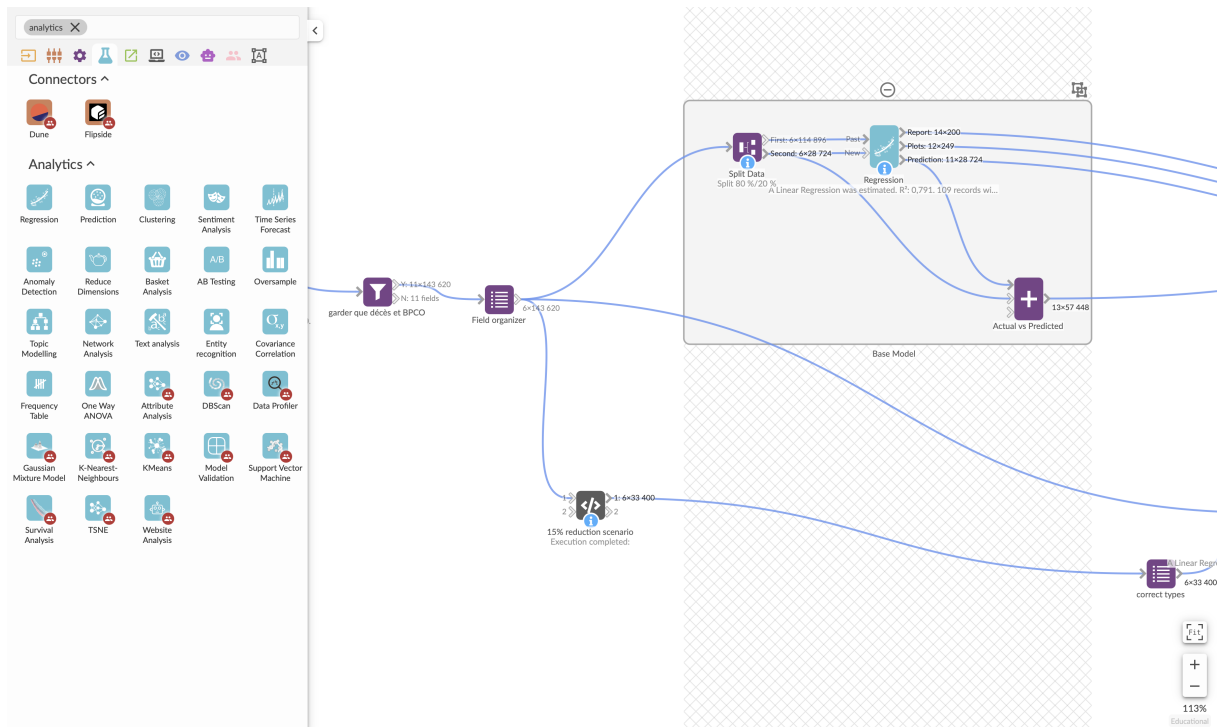


FIGURE 24 – Modèle de régression dans Omniscope

À partir de ces données, nous avons pu utiliser les blocs de modèles prédéfinis dans la partie *Analytics*. Nous avons choisi d'utiliser le modèle appelé *Regression*, car c'était plus ou moins le seul qui correspondait à notre besoin. Ce bloc se base sur du code *R* que nous ne pouvons malheureusement pas inspecter et modifier nous-mêmes. Il propose deux types de régressions : "Regression" et "Adaptive Spline Regression (MARS)". Le premier type propose trois modèles : *Linear Regression*, *Logistic Regression* et *Count Regression*. Nous pouvons cocher les trois et le code se chargera de choisir le meilleur modèle lui-même. Nous avons seulement à choisir notre variable cible et nos variables d'entrée.

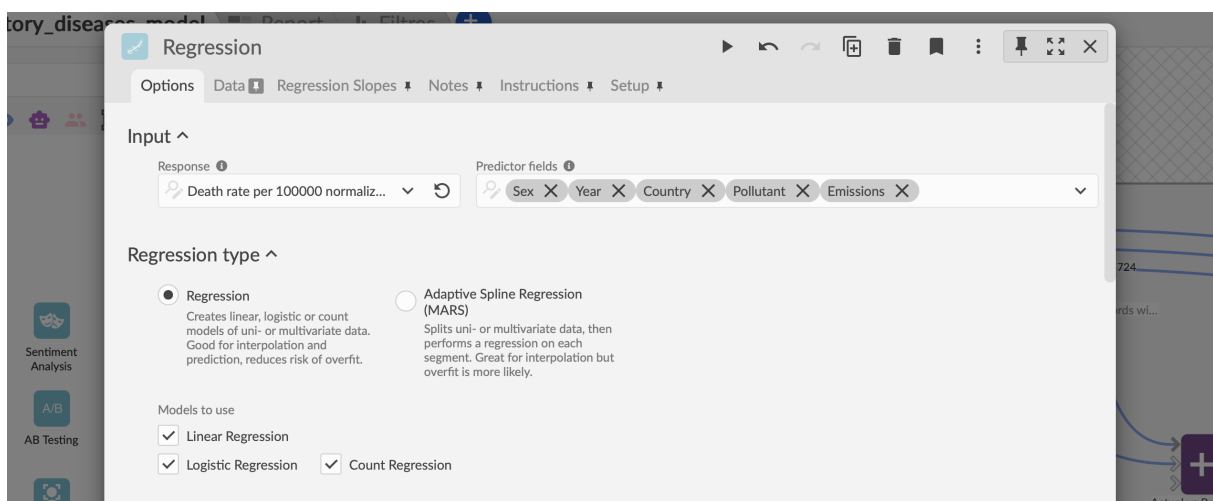


FIGURE 25 – Configuration du modèle de régression dans Omniscope

Nous avons testé les deux solutions. La solution MARS n'a pas fonctionné : la boucle d'entraînement du modèle ne s'est jamais arrêtée, ce qui suggère un problème de convergence que le modèle de Regression classique n'a – a priori – pas. Comme nous n'avons aucun contrôle sur les hyperparamètres, nous nous contenterons du modèle de régression qui fonctionne. Nous lui avons fourni un split 80/20 de nos données pour l'entraînement et le test. Nous obtenons après entraînement un R^2 de 0.79 avec le modèle *Linear Regression*, ainsi que les données brutes de prédiction et des données permettant de visualiser l'impact des variables d'entrée sur la variable cible. Avec les données brutes, nous créons un bloc de concaténation avec les données du split test afin de comparer les prédictions avec les valeurs réelles. Cela nous permet de créer le dashboard suivant :

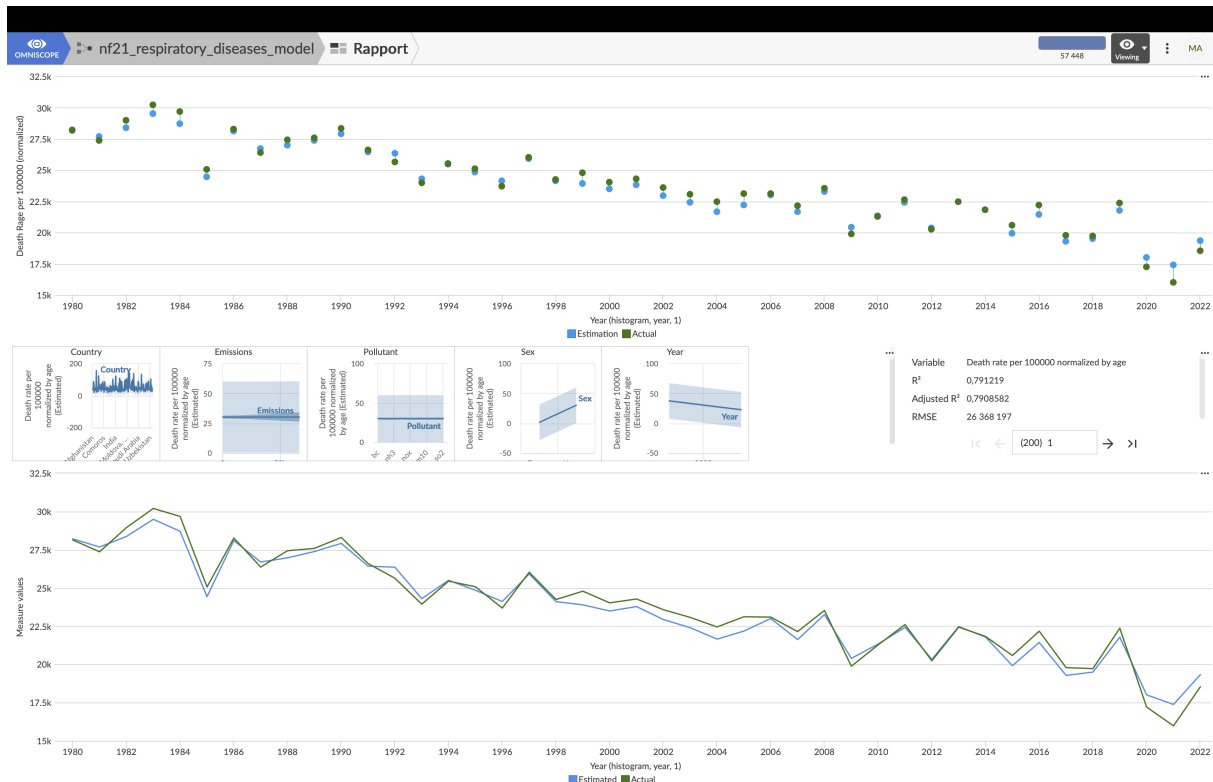


FIGURE 26 – Dashboard Omniscopes des prédictions

On observe en effet que les prédictions sont relativement proches des valeurs réelles. Néanmoins, on reste dans le cadre de nos valeurs de test, et non dans un cadre d'extrapolation comme pour nos scénarii de réduction des émissions. Un problème que nous avons rencontré est qu'avec Omniscopes, il n'est pas possible d'exporter le modèle entraîné pour l'utiliser en dehors de l'outil. Nous ne pouvons donc pas utiliser ce modèle pour faire des prédictions sur nos scénarii de réduction des émissions, ce qui limite grandement l'intérêt de cette implémentation. À la place, nous allons devoir ré-entraîner le même modèle en mettant nos extrapolations comme données de test, ce qui n'est pas optimal car le modèle n'aura pas vu de telles données lors de l'entraînement.

4.3 Prédictions des scenarii dans Omniscope

Nous avons donc ré-entraîné notre modèle de régression dans Omniscope en utilisant nos données de scenarii comme données de test. Afin d'obtenir des données de scenarii, nous avons créé un bloc *Custom Python script* (on aurait tout aussi bien pu utiliser *R*) afin de créer un dataset avec des émissions réduites de 15% sur 10 ans par rapport à l'année de référence.

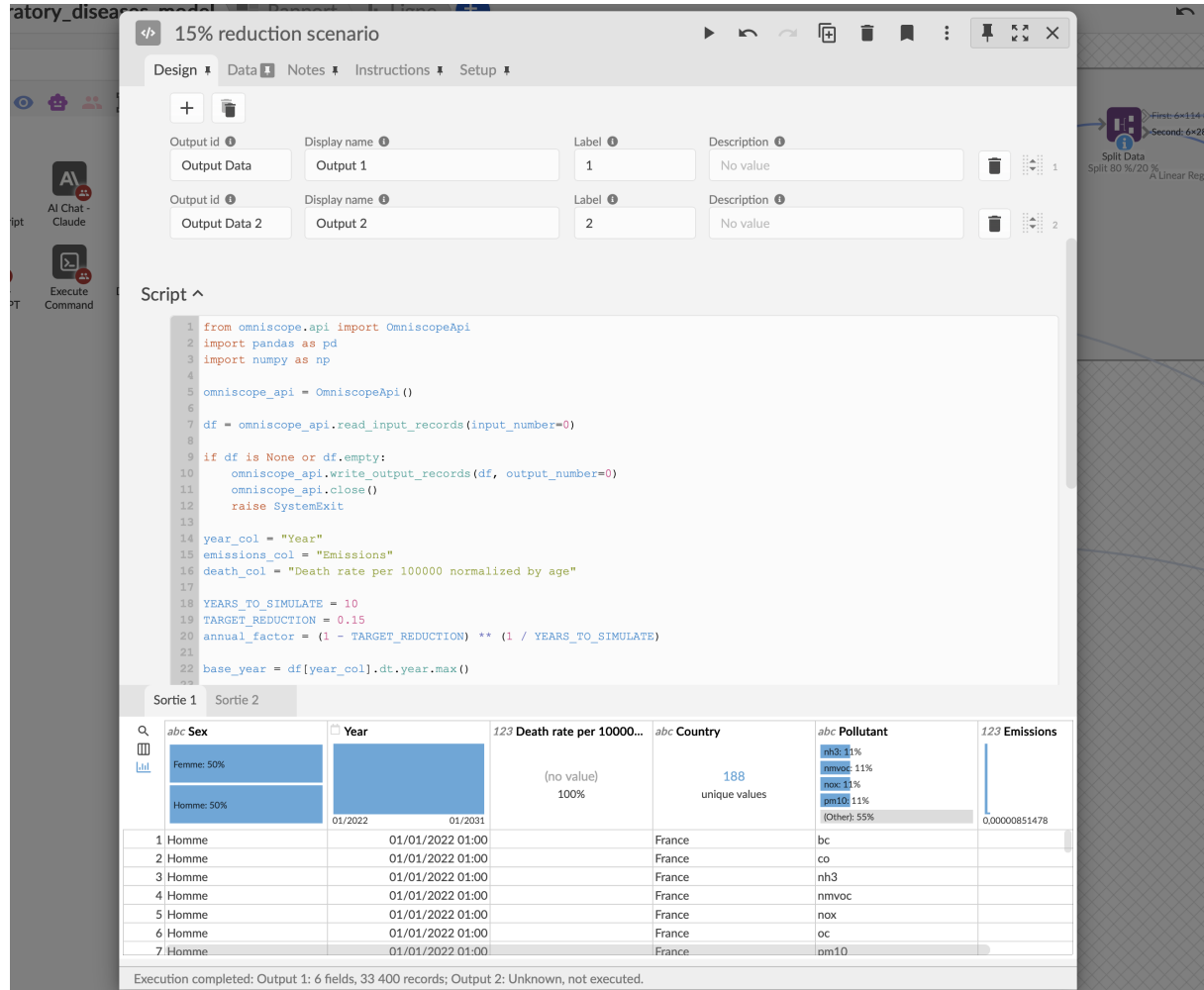


FIGURE 27 – Création des données de scenarii dans Omniscope

Les blocs Python d'Omniscopie sont relativement simples à utiliser, les données en entrée et en sortie sont des DataFrames Pandas, ce qui rend leur manipulation aisée. Nous avons ensuite relié ce bloc au bloc de modèle de régression afin d'obtenir les prédictions sur ces nouvelles données.

Le résultat est le suivant :

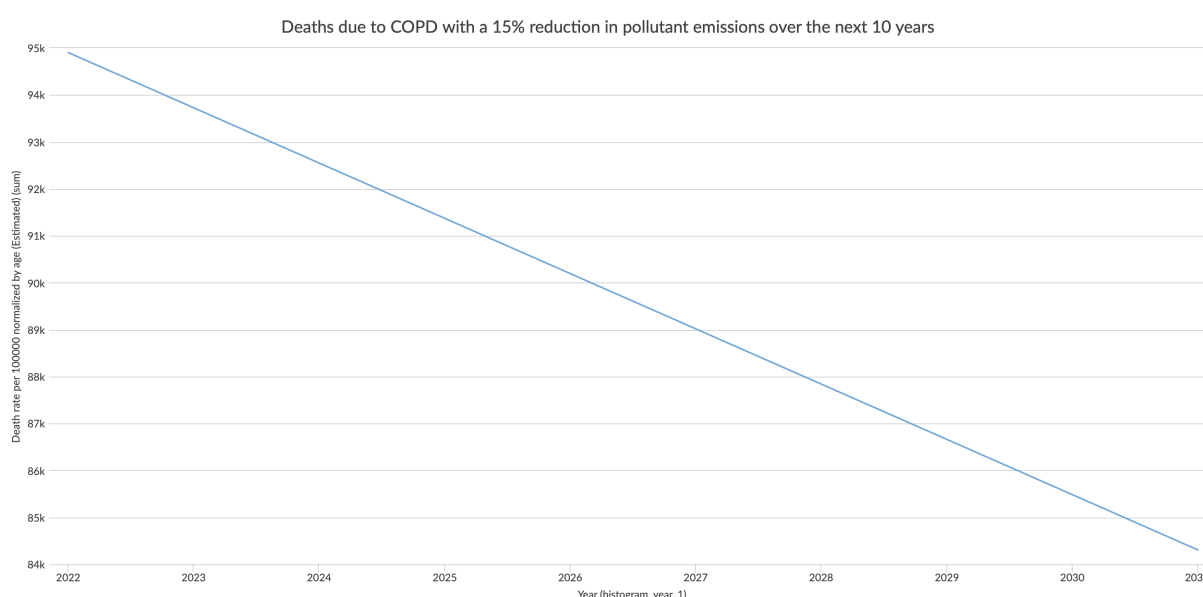


FIGURE 28 – Résultats des prédictions des scenarii dans Omniscope

Le modèle n'a pu prédire qu'une simple baisse linéaire de la mortalité en fonction de la réduction des émissions, ce qui est cohérent avec le modèle de régression linéaire utilisé. Cependant, cette droite semble indiquer plutôt une tendance générale qu'une prédiction précise, car rien ne nous dit que la mortalité due aux BPCO diminue de manière parfaitement linéaire avec la réduction des émissions dans la réalité. Néanmoins, cela illustre bien comment Omniscope peut être utilisé pour implémenter des modèles statistiques et effectuer des analyses de scenarii, même si les capacités sont limitées par rapport à une implémentation complète en Python ou R. La conclusion générale et les interprétations que l'on peut en faire sont les mêmes que pour les modèles précédents : il faut rester prudent dans l'interprétation des résultats et considérer les limites des modèles utilisés.

4.4 Pertinence d'Omniscope pour la modélisation

Omniscope offre une interface conviviale pour la préparation des données et l'implémentation de modèles statistiques, ce qui peut être avantageux pour les utilisateurs non spécialisés en programmation. Cependant, ses capacités de modélisation sont limitées comparées à des bibliothèques dédiées comme scikit-learn ou PyTorch. L'impossibilité d'exporter les modèles entraînés restreint leur utilisation dans des analyses plus complexes ou des scenarii spécifiques. De plus, le choix limité de modèles et l'absence de contrôle fin sur les hyperparamètres peuvent entraver la performance et la flexibilité nécessaires pour des analyses avancées. Ainsi, Omniscope est pertinent pour des analyses exploratoires, des tableaux de bord interactifs, ou encore pour de la modélisation légère. En revanche, pour créer des modèles plus poussés, il nous apparaît comme plus judicieux d'utiliser des outils plus spécialisés, quitte à intégrer ce qui peut l'être dans Omniscope par la suite.

5 Chaînes de valorisation

Deux chaînes de valorisation ont été identifiées pour la phase de modélisation, couvrant l'entraînement des modèles prédictifs et l'analyse de scenarii prospectifs.

5.0.1 Chaîne 5 – Entraînement et évaluation des modèles prédictifs

Objectif : Construire, entraîner et évaluer des modèles de machine learning pour prédire la mortalité respiratoire à partir des données d'émissions de polluants.

TABLE 9 – Chaîne 5 – Entraînement et évaluation des modèles

Tâche générique (TG)	Tâches spécifiques (TS)	Outils
TG51 – Préparation ML	Charger le dataset Parquet, appliquer la transformation log, normaliser avec StandardScaler, séparer train/test (80/20)	Polars, scikit-learn, Omniscope
TG52 – Modélisation classique	Entraîner GradientBoostingRegressor, optimiser les hyperparamètres avec GridSearchCV, extraire l'importance des features	scikit-learn, Omniscope
TG53 – Modélisation deep learning	Définir l'architecture MLP (64→32→1), entraîner avec Adam et MSE loss, ajuster learning rate et epochs	PyTorch
TG54 – Évaluation et sélection	Calculer R^2 et RMSE sur test set, comparer les performances GB vs ANN, sauvegarder les modèles (.joblib, .pt), créer dashboard	scikit-learn, PyTorch, joblib, Omniscope

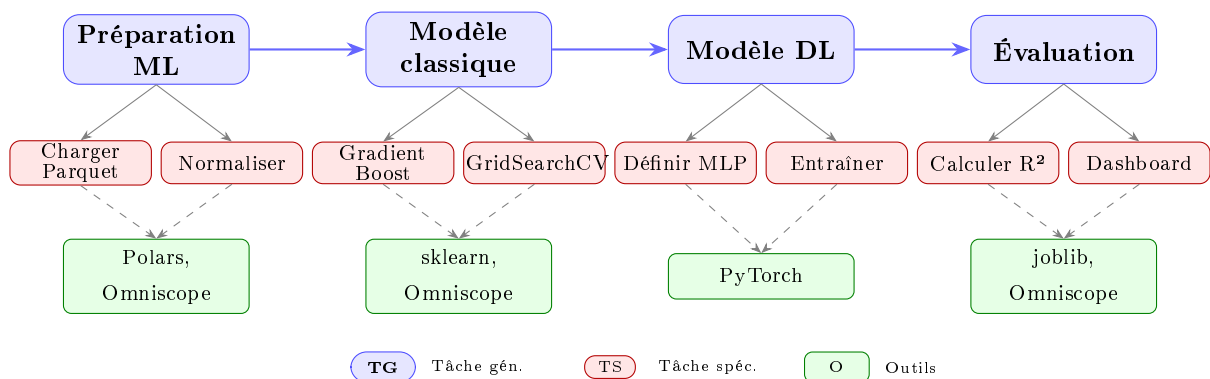


FIGURE 29 – Schéma de la chaîne 5 – Entraînement et évaluation des modèles

5.0.2 Chaîne 6 – Simulation de scenarii de réduction des émissions

Objectif : Exploiter les modèles entraînés pour simuler l'impact de politiques de réduction des émissions sur la mortalité respiratoire et produire des recommandations de santé publique.

TABLE 10 – Chaîne 6 – Simulation de scenarii

Tâche (TG)	générique	Tâches spécifiques (TS)	Outils
TG61 – Définition des scenarii		Établir les niveaux de référence (2022), définir les taux de réduction (5%, 10%, 20%), générer les vecteurs d'émissions simulées	Python (numpy), Omniscope
TG62 – Prédiction prospective		Charger les modèles sauvegardés, appliquer les scalers aux données simulées, effectuer les prédictions pour chaque scénario	scikit-learn, PyTorch, joblib, Omniscope
TG63 – Analyse comparative		Calculer les variations de mortalité prédites, comparer les résultats GB vs ANN, identifier les incohérences d'extrapolation	Python (pandas, numpy)
TG64 – Restitution décisionnelle		Générer les graphiques de prédiction par scénario, synthétiser les recommandations, documenter les limites du modèle	Matplotlib, Omniscope, LaTeX

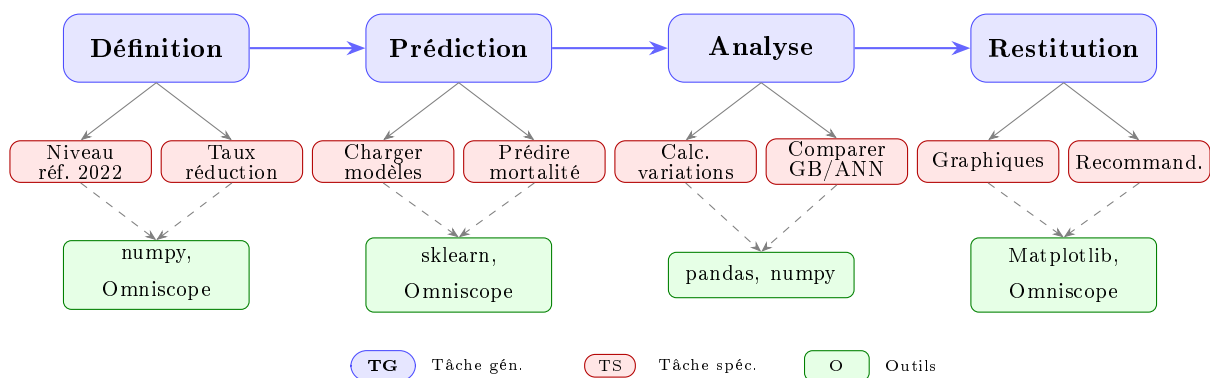


FIGURE 30 – Schéma de la chaîne 6 – Simulation de scenarii

Cinquième partie

Evaluation

1 Évaluation des modèles

L'évaluation rigoureuse d'un modèle de machine learning est une étape cruciale avant toute utilisation opérationnelle. Cette section synthétise les principes généraux d'évaluation et les méthodes appliquées dans notre projet.

1.1 Principes généraux d'évaluation

L'évaluation d'un modèle prédictif repose sur plusieurs axes complémentaires :

1. **Séparation des données** : La première règle fondamentale est de ne jamais évaluer un modèle sur les données ayant servi à l'entraîner. La séparation train/test (typiquement 80/20 ou 70/30, nous avons opté pour 80/20) permet de mesurer la capacité de généralisation du modèle sur des données inédites.
2. **Validation croisée** : Pour des évaluations plus robustes, la validation croisée (k-fold cross-validation) divise les données en k sous-ensembles, entraîne k modèles en utilisant k-1 sous-ensembles pour l'entraînement et 1 pour la validation, puis moyenne les scores obtenus. Cette technique réduit la variance de l'estimation des performances.
3. **Métriques adaptées** : Le choix des métriques dépend du type de problème. Pour la régression, on utilise typiquement :
 - Le **R^2** (coefficient de détermination) : indique la proportion de variance expliquée
 - Le **RMSE** (Root Mean Squared Error) : erreur quadratique moyenne, sensible aux outliers
 - Le **MAE** (Mean Absolute Error) : erreur absolue moyenne, plus robuste aux outliers
4. **Analyse des résidus** : L'examen de la distribution des erreurs permet de détecter des biais systématiques (sous-estimation ou surestimation dans certaines plages de valeurs).

1.2 Méthodes appliquées dans notre projet

Dans le cadre de ce projet, nous avons appliqué les méthodes suivantes :

- **Séparation train/test 80/20** : Le dataset de 8 471 observations a été divisé aléatoirement avec une graine fixe (`random_state`) pour assurer la reproductibilité.
- **Évaluation sur données de test** : Les métriques R^2 et RMSE ont été calculées uniquement sur l'ensemble de test, garantissant une estimation non biaisée des performances.
- **Comparaison de modèles** : Plusieurs algorithmes (Ridge, Random Forest, Gradient Boosting, ANN) ont été comparés sur les mêmes données pour identifier le plus performant.
- **Optimisation des hyperparamètres** : GridSearchCV a été utilisé pour le Gradient Boosting afin de trouver la meilleure combinaison d'hyperparamètres (`n_estimators`, `max_depth`, `learning_rate`).
- **Test d'extrapolation** : Au-delà des métriques classiques, nous avons testé la cohérence des prédictions sur des scénarii hors distribution (réductions d'émissions), révélant les limites de certains modèles.

1.3 Limites de notre évaluation

Plusieurs limitations doivent être soulignées :

- **Pas de validation croisée formelle** : Nous n'avons pas implémenté de k-fold cross-validation, ce qui pourrait légèrement biaiser l'estimation des performances.

- **Données non temporellement séparées** : Le split train/test est aléatoire et non temporel. Pour des prévisions prospectives, un split temporel (entraînement sur années anciennes, test sur années récentes) aurait pu être une option viable.
- **Absence de validation externe** : Nos modèles n'ont pas été validés sur des données provenant d'autres sources, par un soucis de disponibilité, ce qui limiterait la confiance que l'on pourrait avoir si on venait à vouloir généraliser nos résultats.

Sixième partie

Deployment

1 Déploiement du modèle

Note importante : Dans le cadre de ce projet académique, le modèle n'a pas été déployé en production. Cette section présente néanmoins les étapes et considérations nécessaires pour un éventuel déploiement, à titre informatif et pédagogique.

1.1 Sauvegarde et sérialisation du modèle

Avant tout déploiement, le modèle entraîné doit être sauvegardé dans un format permettant son rechargement ultérieur. Deux approches sont couramment utilisées :

- **Pour scikit-learn :** La bibliothèque `joblib` permet de sérialiser les modèles avec `joblib.dump(model, 'model.joblib')`. Le modèle peut ensuite être rechargé avec `joblib.load()`.
- **Pour PyTorch :** Les poids du réseau sont sauvegardés avec `torch.save(model.state_dict(), 'model.pt')`. Il faut également sauvegarder l'architecture du modèle et les paramètres de prétraitement (`scaler`, `transformations`).

1.2 Architecture de déploiement

Plusieurs architectures sont envisageables selon les besoins :

1.2.1 API REST

L'approche la plus courante consiste à exposer le modèle via une API REST. Des frameworks comme **FastAPI** ou **Flask** permettent de créer rapidement un service web :

```
1 from fastapi import FastAPI
2 import joblib
3
4 app = FastAPI()
5 model = joblib.load('model.joblib')
6 scaler = joblib.load('scaler.joblib')
7
8 @app.post("/predict")
9 async def predict(emissions: dict):
10     features = scaler.transform([list(emissions.values())])
11     prediction = model.predict(features)
12     return {"mortality_rate": float(prediction[0])}
```

1.2.2 Conteneurisation Docker

Pour garantir la reproductibilité de l'environnement d'exécution, le service peut être conteneurisé avec Docker. Un Dockerfile typique inclurait Python, les dépendances (`requirements.txt`), le code de l'API et les fichiers du modèle sauvegardé.

1.2.3 Déploiement cloud

Les plateformes cloud offrent plusieurs options :

- **AWS :** Amazon SageMaker pour le ML managé, ou EC2/Lambda pour un contrôle total
- **GCP :** Vertex AI ou Cloud Run pour les conteneurs
- **Azure :** Azure Machine Learning ou Azure Container Instances

1.3 Déploiement de notre modèle

Notre modèle n'a pas été déployé : cela ne fait pas partie du cadre de l'UE NF21, le livrable principal étant ce rapport. Cependant, nous avons présenté les étapes et considérations nécessaires pour un déploiement, à titre informatif et pédagogique.

Septième partie

Conclusion

1 Conclusion générale

Ce projet a permis d’explorer la relation entre les émissions de polluants atmosphériques et la mortalité liée aux maladies respiratoires à l’échelle mondiale. En suivant la méthodologie CRISP-DM, nous avons parcouru l’ensemble du cycle de vie d’un projet de data science : compréhension du contexte métier, exploration des données, préparation, modélisation et évaluation.

Les principaux résultats montrent des corrélations significatives entre certains polluants (NH_3 , OC, SO_2 , PM) et les maladies respiratoires, notamment les BPCO et le cancer du poumon. Les modèles de machine learning développés (Gradient Boosting et réseaux de neurones) atteignent des performances satisfaisantes en interpolation ($R^2 > 0.80$), mais présentent des limites en extrapolation.

Ce travail illustre à la fois le potentiel et les précautions nécessaires lors de l’utilisation de modèles prédictifs pour des décisions de santé publique. Les corrélations observées ne peuvent être interprétées comme des relations causales, et les prédictions prospectives doivent être considérées comme des indications de tendances plutôt que des prévisions précises.

Annexes

Références

- EDGAR v8.0 : <https://edgar.jrc.ec.europa.eu/>
- GBD 2023 : <https://ghdx.healthdata.org/gbd-2023>
- Méthodologie CRISP-DM : https://moodle.utt.fr/pluginfile.php/13371/mod_resource/content/1/CRISP-DM.pdf
- Gradient Boosting : https://en.wikipedia.org/wiki/Gradient_boosting
- Ridge Regression : https://en.wikipedia.org/wiki/Ridge_regression
- Multilayer Perceptron : https://en.wikipedia.org/wiki/Multilayer_perceptron
- scikit-learn API reference : <https://scikit-learn.org/stable/api/index.html>
- Polars API reference : <https://docs.pola.rs/api/python/dev/reference/index.html>

Ressources

- GitHub repository : github.com/malcolm-a/nf21_respiratory_diseases

Remerciements

- **Dr. Babiga BIRREGAH** (enseignant-chercheur UTT) — encadrement et conseils méthodologiques
- **Magali COLIN** (intervenante UTT) — tutoriels Omniscope et support technique
- **Aurélie VAN** (infirmière) — relecture médicale et corrections de la nomenclature